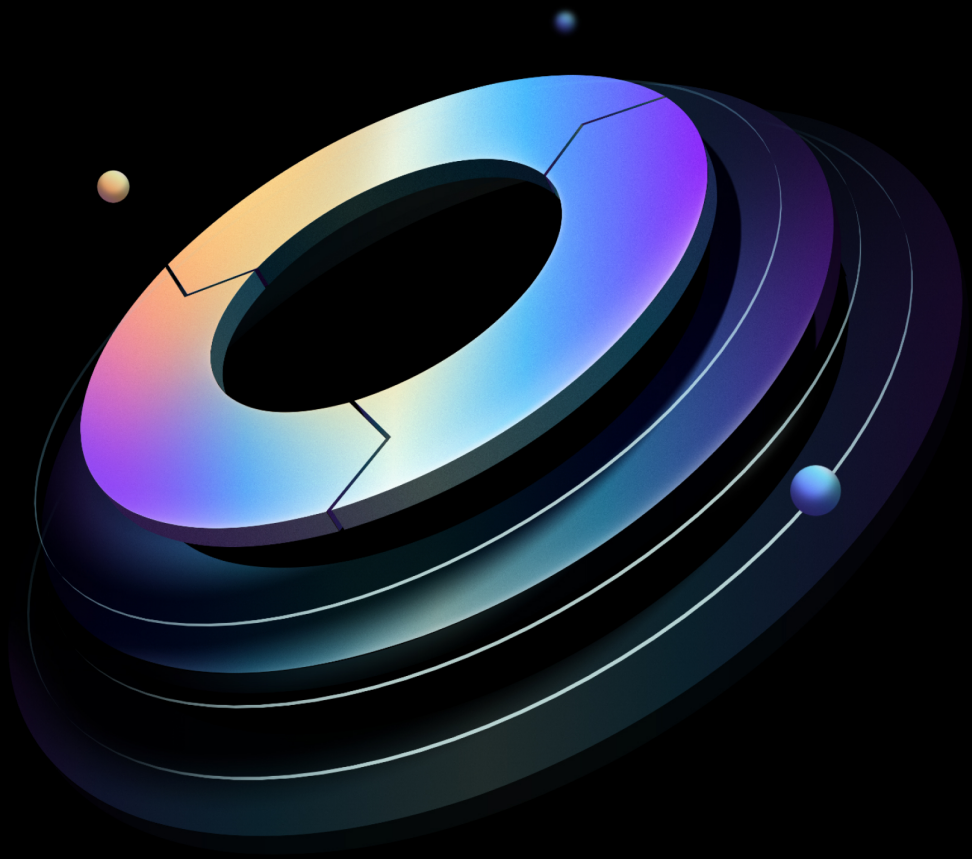
HashiCorp

Terraform Installation Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **7**
1.1 Why use HashiCorp Validated Designs? . . . 7
1.2 Audience . . . 7
1.3 Language and definitions . . . 8

2 Prerequisites and personnel **8**
2.1 Personnel . . . 8
2.2 Access . . . 9

3 Architecture **10**
3.1 Design attributes . . . 10
3.2 Terraform Enterprise components . . . 10
3.3 Operational modes . . . 11
3.3.1 `disk` mode . . . 12
3.3.2 `external` mode . . . 15
3.3.3 `active-active` mode (multiple compute instances) . . . 17
3.4 Operational mode decision . . . 19
3.5 Installation automation . . . 19

4 Detailed design **20**
4.1 Terraform Enterprise application configuration . . . 20
4.2 External components . . . 21
4.2.1 PostgreSQL . . . 21
4.2.2 Redis . . . 22
4.2.3 Object storage . . . 22
4.3 Baseline component sizing . . . 22
4.4 Load balancing . . . 23
4.5 Networking . . . 23
4.6 Observability . . . 24

5 Deployment (VM + container) **24**
5.1 Architectural summary . . . 25
5.2 Terraform modules for installation . . . 26
5.3 Terraform Enterprise license . . . 27

5.4 Process overview . . . 28
5.5 Container-specific guidance . . . 29
5.5.1 General guidance . . . 29
5.5.2 CPU . . . 30
5.5.3 RAM . . . 30
5.6 Platform-specific guidance . . . 31
5.6.1 Resource sizing . . . 31
5.6.2 Cluster calculations . . . 32
5.7 Observability . . . 32

6 Deployment (managed Kubernetes) 33
6.1 Architectural summary . . . 33
6.2 Terraform modules for installation . . . 35
6.3 Process overview . . . 36
6.4 Kubernetes-specific guidance . . . 38
6.4.1 General guidance . . . 38
6.4.2 Use of the internal run pipeline versus HCP Terraform Operator . . . 38
6.4.3 CPU . . . 39
6.4.4 RAM . . . 39
6.4.5 Network . . . 41
6.4.6 Disk . . . 41
6.4.7 Machine sizing . . . 41
6.5 Platform-specific guidance . . . 42
6.5.1 Deployment modules . . . 42
6.5.2 Resource sizing . . . 43
6.6 Observability . . . 44
6.7 Troubleshooting . . . 45
6.7.1 ImagePullBackOff . . . 45
6.7.2 Crash loop back off . . . 45

7 Deployment (manual) 46
7.1 Deployment topology . . . 46
7.2 Component considerations . . . 46
7.2.1 Redis . . . 46
7.2.2 PostgreSQL . . . 47

7.2.3 S3-compatibility . . . 47
7.3 Component sizing . . . 48
7.3.1 Operating system . . . 48
7.3.2 Compute . . . 48
7.3.3 PostgreSQL and Redis . . . 49
7.4 Observability . . . 49
7.5 Networking requirements . . . 49
7.6 Installation process - Docker . . . 50
7.6.1 Prepare requirements . . . 50
7.6.2 Tasks . . . 50
7.6.3 Create a Docker Compose configuration . . . 51
7.6.4 Configure systemd . . . 52
7.6.5 Download the Terraform Enterprise image . . . 52
7.6.6 Run Docker Compose . . . 53
7.6.7 Run a health check . . . 53
7.6.8 Retrieve your initial administrator creation token . . . 55

8 Air-gapped configuration 58
8.1 Network egress . . . 58
8.2 Five components to stage internally . . . 60
8.3 Application container image . . . 61
8.3.1 Staging the image . . . 61
8.3.2 Configuring TFE to use the internal registry . . . 61
8.3.3 TLS and proxy requirements . . . 62
8.3.4 Terraform run pipeline (optional) . . . 62
8.4 Client binaries . . . 63
8.4.1 Registering versions in Terraform Enterprise . . . 63
8.5 Sentinel CLI binaries . . . 64
8.6 Provider binaries . . . 65
8.6.1 Option 1 — Network mirror protocol . . . 65
8.6.2 Option 2 — Terraform Enterprise private registry . . . 65
8.6.3 Choosing between the two options . . . 66
8.7 Additional egress considerations . . . 66
8.7.1 SAML and single sign-on . . . 66
8.7.2 VCS integration . . . 66

8.7.3 HCP Terraform agents . . . 67
8.7.4 Custom agent images . . . 70
8.7.5 License validation and reporting . . . 70
8.7.6 Cost estimation . . . 70
8.7.7 Summary of egress dependencies . . . 70
8.8 Pre-baked VM images with Packer . . . 72
8.9 Ongoing operational commitments . . . 73
8.10 Additional considerations . . . 74

9 Post-installation configuration 74
9.1 Post-installation tasks . . . 75
9.2 Connect to a version control system . . . 75
9.2.1 GitHub app . . . 75
9.2.2 OAuth . . . 76

10 Identity integration 76
10.1 SSO and teams . . . 76

11 Security hardening 77
11.1 Operating system . . . 77
11.2 Application . . . 77

12 High availability and multi-region 78
12.1 Primary considerations . . . 79
12.1.1 Cost versus risk . . . 79
12.1.2 Cloud support . . . 79
12.1.3 Automation . . . 79
12.1.4 Testing . . . 80
12.1.5 Configuration security . . . 80
12.2 Component-specific guidance . . . 81
12.2.1 Compute . . . 81
12.2.2 S3-compatible storage . . . 84
12.2.3 PostgreSQL . . . 88
12.2.4 Redis . . . 90

13 Observability setup **90**
13.1 Configuring data collection . . . 90
13.1.1 Terraform Enterprise metrics and logs . . . 91
13.1.2 HCP Terraform audit trail logs . . . 93
13.1.3 Metrics and logs on HCP Terraform agents . . . 93

14 Backup and restore initial setup **95**
14.1 Definitions . . . 95
14.2 Best practices . . . 96
14.2.1 Maintain the backup and restore process . . . 96
14.2.2 Manage sensitive values . . . 96
14.2.3 Process audit logs . . . 97
14.2.4 Terraform Enterprise backup API . . . 97
14.3 Initial considerations . . . 97

15 Disaster recovery initial setup **98**
15.1 Detection of need . . . 98

16 Next steps **98**
16.1 Product support . . . 98

17 Appendix **99**
17.1 AWS IAM policy . . . 99

18 Changelog **102**
18.1 2026-08 . . . 102
18.2 2026-01 . . . 102
18.3 2025-12 . . . 102
18.4 2025-11 . . . 103
18.5 2025-06 . . . 103
18.6 2025-05 . . . 103
18.7 2025-04 . . . 103
18.8 2025-03 . . . 103
18.9 2025-02 . . . 103
18.10 2025-01 . . . 103
18.11 2024-04 . . . 104

18.122023-10 104

19 Credits **104**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This guide is for platform engineers, infrastructure architects, DevOps administrators, and cloud operators who design and deploy Terraform Enterprise in cloud, datacenter, or private-cloud environments. After deployment, operators should use the Terraform Administration Guide for day-2 operation of Terraform Enterprise and HCP Terraform, and practitioners should use the Terraform User Guide for infrastructure provisioning workflows.

Note

HCP Terraform is a SaaS service operated by HashiCorp and does not require installation by customers. HCP Terraform readers should start with the Terraform Administration Guide for shared service operations and the Terraform User Guide for consumption workflows.

1.3 Language and definitions

HashiCorp is an enabler of multi-cloud strategies, and as such we take this into account when writing designs. While we aim to use technology-agnostic terminology, the document primarily supports the three largest cloud service providers (CSPs) together with an on-premise/datacenter architecture. There are some terms which do not translate between the public cloud and the data-center. For the sake of clarity, our definitions for these terms are below.

Term	Definition
Availability zone	A separate failure domain within a logical datacenter.
Region	A separate logical datacenter.
Public subnet	A network accessible from the public Internet, containing publicly addressable infrastructure.
Private subnet	A network not accessible *from* the public Internet and whose infrastructural objects are either blocked from connecting *to* the public Internet or do through a NAT gateway.

2 Prerequisites and personnel

This section defines the roles required from the participants, and includes authority and access needs to the working environment.

2.1 Personnel

Focusing on the deployment activity alone, we recommend enlisting a project leader and a cloud administration team. A project leader coordinates events, facilitates resources and assigns duties to the Cloud Administration Team. Members of the Cloud Administration Team carry out functional tasks to install Terraform Enterprise. For cloud administrators, we make assumptions about general knowledge of the following items.

- Cloud architecture and administration
- Administration-level experience with Linux
- Practical knowledge of Docker
- Practical knowledge of Terraform

In addition, we strongly recommend requesting assistance from a security operations team. The emphasis is on integrating formal security controls required for services hosted in your preferred cloud environment. It is also essential to specify a role for the production services team who take over the deployment when the project goes live.

2.2 Access

- The installation team requires direct access (including administrator) to the following components to install and configure Terraform Enterprise.
    - Compute/storage instances
    - Network objects such as firewall rules, load balancers
    - Certificates: TLS certificate material
    - Identity such as AWS IAM, GCP Cloud Identity, or AAD
    - Secrets management and associated key management services: AWS Secrets Manager, AWS KMS, GCP Secret Manager, GCP Cloud Key Management, Azure KeyVault, VMware vSphere Native Key Provider
- License File: To deploy Terraform Enterprise you must obtain a license from HashiCorp.
- Certificate authority:
    - Terraform Enterprise requires a TLS certificate and private key on each node. This certificate must match the Terraform Enterprise hostname, either using the FQDN or being a wildcard certificate.
    - Sign the certificate with a public or private CA.
    - The key and X.509 TLS certificate must be PEM encoded. Terraform Enterprise validates the certificate to ensure it uses a Subject Alternative Name (SAN) for Domain Names (DN) entries and not just a Common Name (CN) entry.
- DNS: Ensure that a DNS record exists for Terraform Enterprise which matches the SAN in the certificate.

3 Architecture

Terraform Enterprise automates infrastructure provisioning and lifecycle management. It also integrates with the ecosystem to offer additional features like single sign-on for user authentication, version control, observability, and artifact management.

3.1 Design attributes

Most decision points in this document require the evaluation of non-functional requirements to find a recommendation that is acceptable within the bounds of a typical customer environment. Where possible, decisions accompany associated narrative allowing the reader to understand our rationale.

- **Availability**: Represents design points that minimize the impact of subsystem failures on the solution uptime. *An example of this is introducing multiple load-balanced application instances.*
- **Operational complexity**: From time to time, it is necessary to introduce some complexity during the initial deployment to provide a reduced effort for operations teams to manage the solution. *An example of this is introducing a machine imaging process with Packer that permits immutable operations for scaling up/down and application updates.*
- **Scalability**: Some design choices may work at a small scale but introduce overhead as the solution scales. *An example of this is choosing to onboard teams using a manual process in the Terraform Enterprise UI rather than creating an automated onboarding workflow to complete the necessary setup.*
- **Security**: Indicates how a decision changes the security posture of the overall solution. *An example of this is taking advantage of workload identity so that workspaces authenticate against their API targets using JWT based workflows rather than using long-lived credentials as workspace variables.*

3.2 Terraform Enterprise components

Terraform Enterprise consists of several components, each with specific responsibilities in the overall architecture. This section explains the function of each component and the technologies they support.

- **Terraform Enterprise application**: This is the Terraform Enterprise container that HashiCorp provides as the core application. See the Terraform HVD Module code for the latest default machine size or type for the respective cloud or Kubernetes environment. These link from the *Deployment* pages later in this HVD.
- **HCP Terraform agent (optional)**: A fleet of isolated execution environments that perform Terraform runs on behalf of Terraform Enterprise users. The HCP Terraform agent (which is also usable with Terraform Enterprise) can consume APIs in isolated network segments to provide quality of service to specific teams in your organization or distributes load away from the core Terraform Enterprise service. While use of HCP Terraform agents is highly recommended, it is not required. Terraform Enterprise comes with built-in agents for job execution.
- **PostgreSQL database**: Serves as the primary store of Terraform Enterprise's application data such as workspace settings and user settings. You can review the supported versions and requirements on our Terraform Enterprise PostgreSQL page.
- **Redis cache**: Used for caching and coordination between Terraform Enterprise core's web and background workers. In order to have multiple, simultaneously active application nodes, you require an external cache. We have validated Terraform Enterprise with the native Redis services from AWS, Azure, and GCP. Terraform Enterprise uses Redis internally on single-node VMware-based deployments. You can review the supported versions and requirements on our Terraform Enterprise Redis page.
- **Object Storage**: Used for storage of Terraform state files, plan files, configuration, output logs and other such objects. All objects persisted to blob storage are symmetrically encrypted then written. A unique encryption key encrypts each object using 128 bit AES in CTR mode. The internal HashiCorp Vault transit secret engine processes the key material, which stores the default transit encryption cipher (AES-GCM with a 256-bit AES key and a 96-bit nonce), alongside the object. This pattern is envelope encryption. Any S3-compatible object storage service, GCP Cloud Storage or Azure blob storage meets Terraform Enterprise's object storage requirements. You must create an object store instance for Terraform Enterprise to use, and specify its connection string details during installation. We recommend you ensure the object store is in the same region as the Terraform Enterprise instance.

3.3 Operational modes

In this section, we review the supported operational modes for Terraform Enterprise.

From v202309-1, Terraform Enterprise is compatible with a variety of platforms including Kuber-

netes. Our official installation documentation for this is here. We recommend using this for all private cloud deployments.

Operational mode	Characteristics	Description
`disk`	All Terraform Enterprise services deploy onto a single node.	Most suitable for functional testing, training, and gaining familiarity. Do not use for production.
`external`	External database, external object storage, stateless application installed on a single node with internal cache with optional, remote Terraform agents.	Most suitable for organizations who focus on system performance, but less so with system resilience.
`active-active`	Replicated database and object storage, external caching layer and multiple, stateless application nodes in redundant availability zones. Optionally, multiple remote Terraform agents.	Most suitable for a business-wide service that requires the highest levels of resilience and performance.

3.3.1 `disk` mode

In a `disk` mode deployment, an organization deploys Terraform Enterprise in a singular compute environment without failover or active/active capabilities. In this deployment model, all services (including PostgreSQL DB and other storage requirements) are self-contained within the Docker environment, eliminating external dependencies. The application data is self-managed and uses localized Docker disk volumes for caching and data operations.

A `disk` deployment is well-suited for development, functional testing, and acceptance testing scenarios, however we do not recommend that you use this for any production environments. For

production deployments, use only the `external` or `active-active` operational modes.

The following diagram illustrates the logical architecture of a *`disk`* Terraform Enterprise deployment:

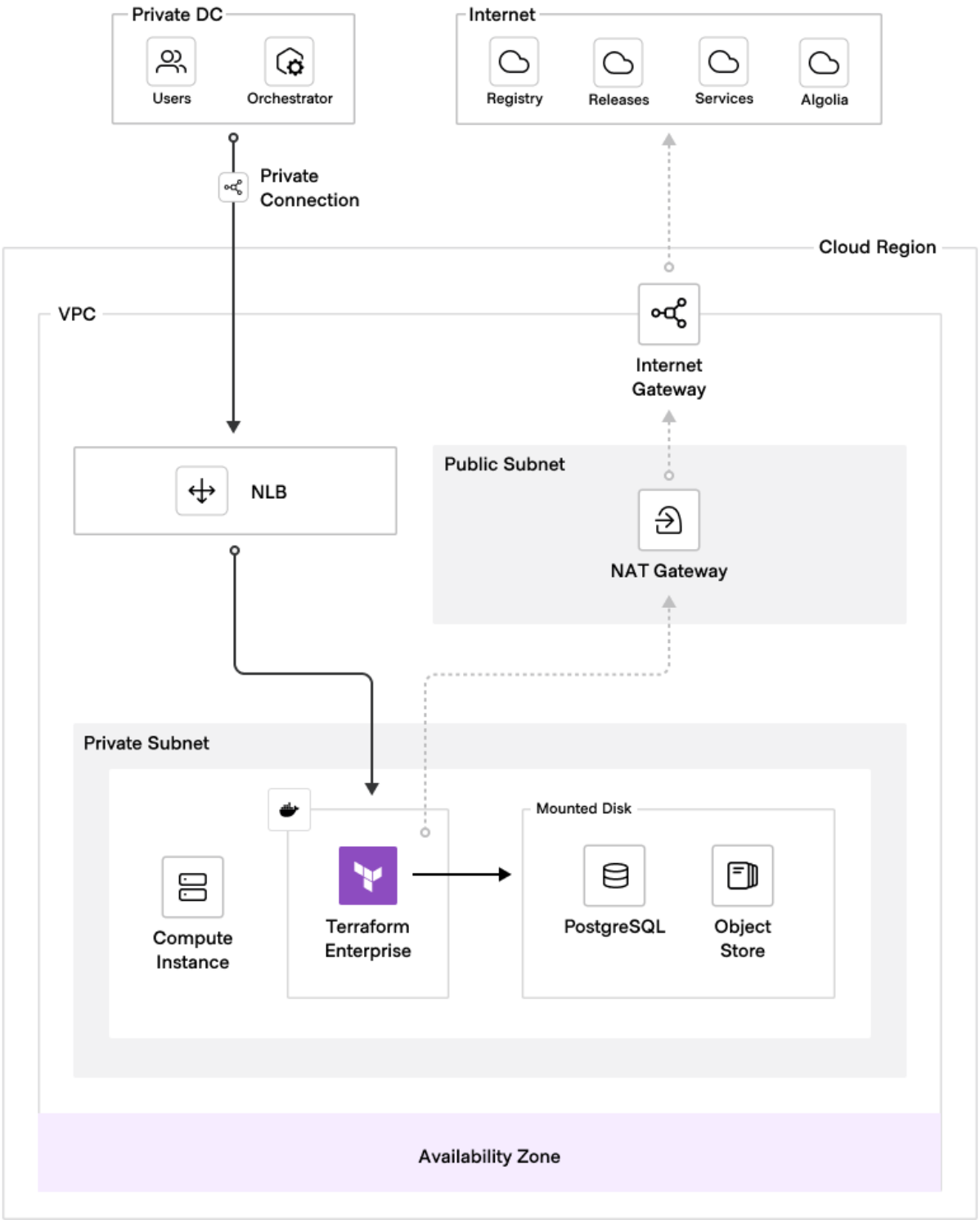


Figure 1: Terraform disk operational mode

The benefits of this deployment model include minimal resource requirements, reduced reliance on specialized expertise, and rapid deployment. However, it lacks enterprise-grade resiliency for unexpected downtime and cannot scale performance without interrupting services.

3.3.2 `external` mode

To move to a stateless application front-end, the `external` operational mode takes the application database and object storage and moves them to dedicated infrastructure. The cache remains internal but represents transient caching only.

Each of the public clouds provides native services for a PostgreSQL database and object storage that supports this deployment pattern. The configuration of these external services decouples the application infrastructure, eliminating single points of vulnerability and improving availability. While this deployment model may not offer performance scalability (although AWS Aurora storage scales and minimizing downtime is possible when scaling the DB instance through instance failover), it enhances the deployment resilience by distributing the core components.

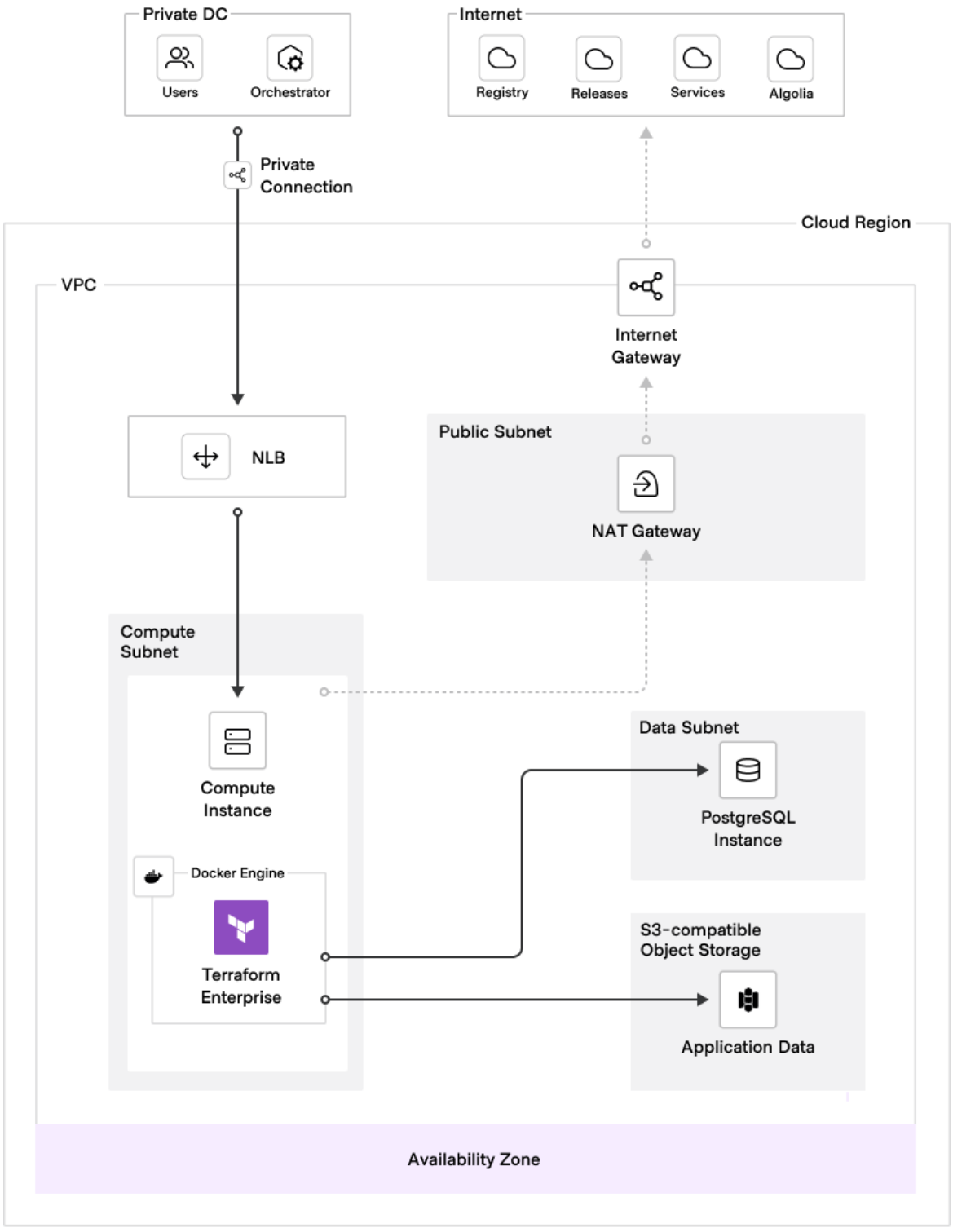


Figure 2: external mode operational mode

The model is an initial step towards a fully resilient and scalable deployment. It relies on one compute environment for a Docker engine. This means that this single instance of the Terraform Enterprise application becomes a vulnerable single point of failure for an enterprise-grade operation. If your target architecture requires zero downtime from expected failures such as instance or availability zone failure, we recommend `active-active` below.

3.3.3 `active-active` mode (multiple compute instances)

When your organization requires increased availability or performance from Terraform Enterprise that your current single application instance cannot provide, it is time to scale to the `active-active` operational mode.

Before scaling to this mode, weigh its benefits against the increased operational complexity. Specifically, consider the following aspects.

- An automated Terraform Enterprise deployment process is a hard requirement
- When monitoring multiple Terraform Enterprise instances, it is important to consider the impact on your monitoring dashboards and alerts.
- To effectively manage the lifecycle of application nodes, custom automation is necessary.

Terraform Enterprise supports `active-active` mode, which involves combining multiple instances of Terraform Enterprise with external services. In this model, separate Docker engines (on VMs, or by an orchestrator such as Kubernetes) host the primary application services, spread across at least three availability zones within the same cloud region. These instances connect to centralized services that provide the application database, shared object storage and durable cache, all managed by the cloud service provider.

Using an active/active model has a key advantage: It eliminates potential service failure points. By distributing multiple application instances, the service becomes more resilient and available. The deployment includes redundant elements that improve reliability, increase resilience, and enhance performance.

The following diagram illustrates the recommended architecture for an `active-active` mode deployment. It provides a visual representation of data flow, user access, key connections, and infrastructure components.

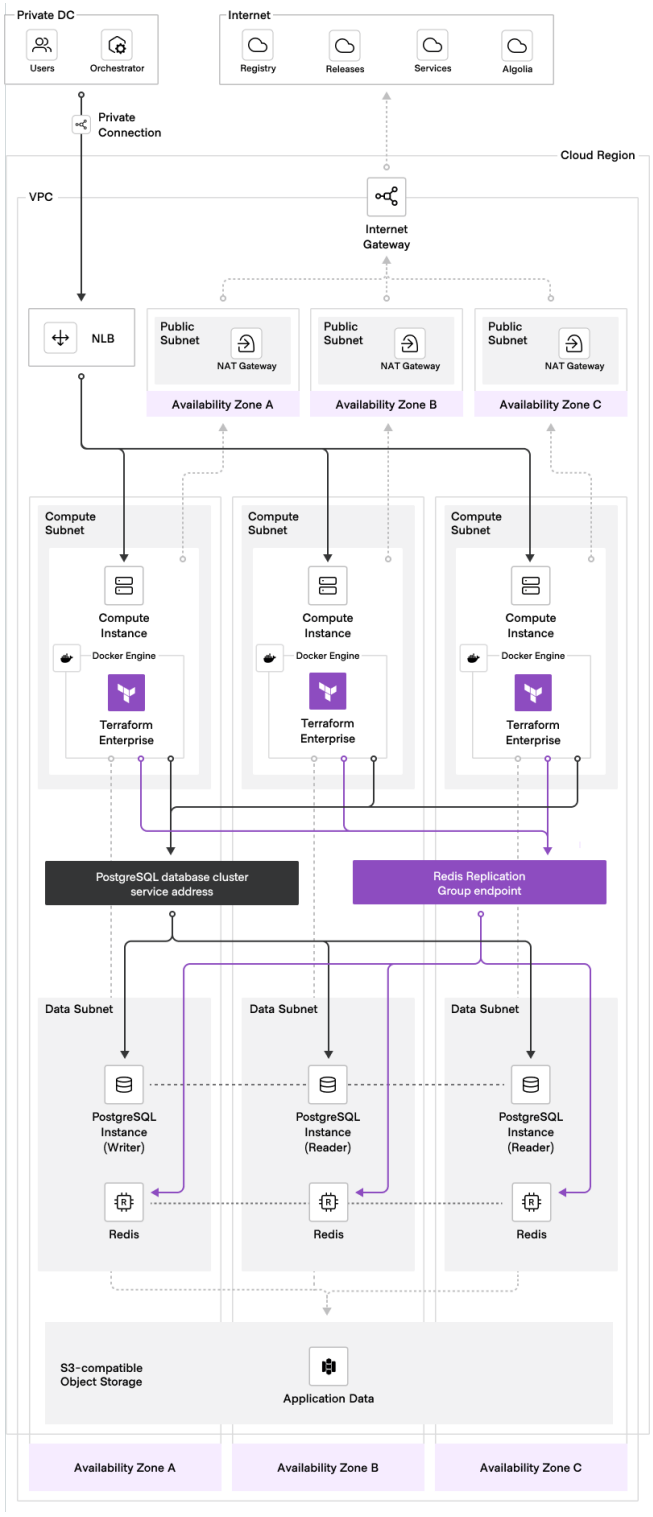


Figure 3: active-active mode operational mode

In addition to improved availability, `active-active` mode offers scalability and supportability benefits. The presence of multiple application instances allows for better workload distribution, alleviating bottlenecks and enhancing overall performance, with redundant application instances and external services.

> **Note**
>
> Please note that Redis does not have to be external when there is a single node of Terraform Enterprise. When using our Terraform-based deployment modules, they provision a separate Redis instance automatically when the active/active parameter is **`true`**. For HVD Module deployments onto any managed Kubernetes service, the module also deploys a required Redis instance as an external service.

3.4 Operational mode decision

The `active-active` mode topology offers crucial benefits to your organization and is our recommendation because it:

- Ensures service availability and scalability while mitigating potential failure scenarios caused by service interruption or resource starvation.
- Safeguards against revenue loss due to unscheduled service interruptions.
- Ensures data integrity while adhering to your data residency security guidelines.

Your choice of operational mode also constrains how you install Terraform Enterprise. Deploying in `active-active` mode makes an automated deployment process a hard requirement, so factor this into your installation approach as you decide on a mode.

3.5 Installation automation

Regardless of the operational mode you select, we recommend that you automate the deployment of Terraform Enterprise. Where your organization already has a configuration management standard — for example Chef, Puppet, Ansible, or Salt — we recommend building on it, because it provides:

- A well-known approach to installation and software management for your organization, and therefore a lower barrier to a successful deployment.

- Adherence to an organizational standard, which also increases the likelihood of regulatory compliance where applicable.

We advise automating the deployment of Terraform Enterprise irrespective of the operational mode used because:

- Manual installation increases the time and human involvement required to upgrade each instance every month. Automation reduces this burden.
- Automated installation lets you version and collaborate on the code used.
- Automated installation lets you rely on the same deployment pattern every time, which increases maintenance confidence for both the architecture teams and the services teams responsible for managing the product in production.

The tooling and modules that implement this automation are platform-specific. See the relevant *Deployment* page for platform-specific installation recommendations, including the HashiCorp-maintained modules for each target.

4 Detailed design

The architecture page describes the Terraform Enterprise software building blocks and the operational modes that arrange them. This page goes a level deeper into the software components and configuration that are **common to every deployment**, regardless of the platform you run on.

Use this page to understand the external components Terraform Enterprise depends on, the baseline resources those components require, and the cross-cutting concerns — such as load balancing and networking — that every deployment must satisfy. The platform-specific implementation of these concerns, including cloud instance types and managed-service SKUs, lives on the relevant *Deployment* page.

4.1 Terraform Enterprise application configuration

The following software configuration decisions apply to every Terraform Enterprise deployment. The mechanism you use to set them varies by platform (environment variables in a Docker `compose.yml`, Helm overrides on Kubernetes, and so on), but the decisions themselves are common.

- **Encryption and the embedded Vault**: Terraform Enterprise uses an embedded instance of HashiCorp Vault for data-security operations. The `TFE_ENCRYPTION_PASSWORD` decrypts the Vault token stored in the database, and Terraform Enterprise then uses that token for all subsequent encryption and decryption. Treat this value as a sensitive company secret and generate it in line with your corporate password policy.
- **Run concurrency**: `TFE_CAPACITY_CONCURRENCY` controls the number of concurrent workspace runs per Terraform Enterprise instance (default `10`), and each concurrent run reserves memory (default 2 `GB`, set by `TFE_CAPACITY_MEMORY`). These values drive the compute sizing for both the application and the execution layer, so decide them before you size any platform. The per-platform sizing calculations that build on these defaults are on the *Deployment* pages.
- **TLS**: Terraform Enterprise terminates TLS on the application. Every deployment requires a server certificate and private key whose DNS SAN includes the FQDN that resolves to the load balancer. We do not recommend self-signed certificates in any environment. Terraform Enterprise cannot use passphrase-protected private keys.

4.2 External components

In any production operational mode (`external` or `active-active`), Terraform Enterprise depends on three external components. These are software requirements common to all platforms; how you provision them — self-managed or through a Platform-as-a-Service (PaaS) offering — is a platform decision covered on the *Deployment* pages.

4.2.1 PostgreSQL

PostgreSQL is the primary store of Terraform Enterprise application data, such as workspace and user settings. Terraform Enterprise requires PostgreSQL version 13.x or later; we recommend a version later than 15.x for new installations because some 14.x versions are not supported. There are also specific schema requirements documented at that link, so involve your DBA team early so that all parties understand the requirements at the outset.

4.2.2 Redis

Redis provides caching and coordination between the web and background workers. An external Redis instance is a hard requirement whenever more than one Terraform Enterprise application node is active — that is, for `active-active` mode. Terraform Enterprise uses Redis internally on single-node deployments, so `disk` and single-node `external` deployments do not require a separate Redis instance. Supported versions are 6.2 and later, and version 7, which we recommend. Redis Sentinel is not supported.

4.2.3 Object storage

Object storage holds Terraform state files, plan files, configuration, and output logs. Any S3-compatible object store, Google Cloud Storage, or Azure Blob Storage meets the requirement. All objects are encrypted before they are written: each object is encrypted with a unique key using 128-bit AES in CTR mode, and the embedded Vault transit secrets engine wraps that key material (AES-GCM with a 256-bit key and a 96-bit nonce) alongside the object — a pattern known as envelope encryption. Object store consumption grows with both the number of workspace runs and the size of the configuration directories submitted to the API, so keep configuration repositories limited to the files a run genuinely needs.

4.3 Baseline component sizing

The following baseline is a **vendor-agnostic minimum specification** for the Terraform Enterprise application and its external components. It is appropriate for most initial production deployments and for development or testing environments. Treat it as the target that every deployment must meet or exceed, whichever platform you choose:

- A **self-managed** deployment (for example a manual private-cloud install) provisions PostgreSQL and Redis to these specifications directly.
- A **PaaS** deployment (for example AWS Aurora and ElastiCache, or the Azure and GCP equivalents) selects a managed-service tier that meets or exceeds these specifications.

Component	vCPU / cores	Memory	Disk
Terraform Enterprise application (per node)	4 vCPU	32 GB	1 TB
PostgreSQL	4 core	32 GB	2 TB
Redis	4 core	16 GB	500 GB

Configure CPU and memory identically across Terraform Enterprise application nodes. Many scaled customers begin with 8 vCPU and 32 GB RAM for the application as an initial production specification, and expect to scale past the baseline as adoption grows. Disk backing the application must sustain at least 3000 IOPS, and the Terraform Enterprise application requires at least 40 GB of available space (for Docker deployments this must be available to `/var/lib/docker`; the limit does not apply to Kubernetes).

The mapping of this baseline to concrete cloud instance types and managed-service SKUs is platform-specific and is documented in the resource-sizing table on each *Deployment* page.

4.4 Load balancing

Every production deployment places a **layer 4 (TCP) load balancer** in front of the Terraform Enterprise application nodes. A layer 4 load balancer is required because certificates must be present on the compute nodes for Terraform Enterprise to operate, and terminating TLS on the application — rather than at the load balancer — is more secure and avoids re-encrypting traffic behind the load balancer. This design decision is common to all platforms; the specific load balancer resource and its configuration are covered on each *Deployment* page.

4.5 Networking

Terraform Enterprise has a defined set of ingress and egress requirements that apply to every deployment. Refer to the network requirements page for the authoritative list. In summary:

- We do not recommend exposing Terraform Enterprise to the public Internet; users should reach the API and UI from the company network.
- Terraform Enterprise typically requires egress to the HashiCorp container registry and to HashiCorp service APIs (for example `registry.terraform.io`, `releases.hashicorp.com`,

and `reporting.hashicorp.services`), plus any VCS, SAML, and cost-estimation endpoints your use case needs.
- Where outbound access is restricted, Terraform Enterprise can run fully air-gapped by hosting provider and Terraform binaries in its own registry.

The application architecture is single-region. How you implement these requirements — subnets, security groups, egress controls, and air-gapping — is platform-specific and is covered on each *Deployment* page.

4.6 Observability

Terraform Enterprise exposes metrics and logs that every deployment should collect. The setup of this software and general-component observability — enabling metrics, forwarding logs and audit trails, and collecting agent telemetry — is covered in Observability setup. Platform-specific observability, such as the log and metric destinations native to each cloud, is covered in an observability section on the relevant *Deployment* page. Ongoing, day-2 observability and alerting belongs in the Administration Guide.

5 Deployment (VM + container)

This page provides guidance on how to deploy Terraform Enterprise using Docker on a VM running on AWS, Azure, or GCP using Terraform modules written and managed by HashiCorp (HVD Modules). HashiCorp designs the modules for use in conjunction with the Terraform command-line tool. The modules are available in the HashiCorp Terraform Registry and provide the best practice method for deploying Terraform Enterprise on the respective cloud.

We expect that the reader has already reviewed the architecture page of this HVD and has a good understanding of the Terraform Enterprise architecture. If you have not done so, we recommend you review the architecture page before proceeding with the deployment. We also expect that the reader has a good understanding of Docker or Podman, and of deploying and managing virtual machines on public clouds.

5.1 Architectural summary

- Terraform Enterprise deploys on a VM running on AWS, Azure or GCP, irrespective of the deployment topology. If using Active/Active, you scale additional VMs, running them in different availability zones (AZs) in the same region.
- Deploy one or more Terraform Enterprise container(s) onto respective VMs depending on your chosen deployment topology.
- Use managed versions of object storage, PostgreSQL database. If using Active/Active, use a managed Redis cache (as offered by your public cloud, specifically not Redis Cluster), to ensure the system distributes replicas in different AZs.
- Use a layer 4 load balancer to ingress traffic to the Terraform Enterprise instances. This is because:
    - Certificates need to be on the compute nodes for Terraform Enterprise to work.
    - It is more secure to terminate the TLS connection on Terraform Enterprise rather than outside it and have to re-encrypt traffic from the inside of the load balancer requiring an additional certificate.
- By using three AZs (one Terraform Enterprise pod in each AZ), the system has an n-2 failure profile, surviving failure of two AZs. However, if the entire region is unavailable, then Terraform Enterprise is unavailable. The application architecture is single-region.
- We do not recommend that you expose Terraform Enterprise for ingress to the public Internet. Users must be on the company network to be able to access the Terraform Enterprise API/UI. However, we recommend Terraform Enterprise is allowed to access certain addresses on the Internet to access:
    - The HashiCorp container registry - where HashiCorp vends the Terraform Enterprise container from
    - HashiCorp service APIs (all owned and operated by HashiCorp except Algolia):
        - `registry.terraform.io` houses the public Terraform module registry which enterprise customers want to avoid letting users have unfettered access to as it contains community-contributed modules and providers uncertified by HashiCorp. However, this is where official providers index.
        - `releases.hashicorp.com` is where HashiCorp host Terraform binary releases. We recommend users stay within two minor releases of latest to access the latest security updates, and new features.

        - `reporting.hashicorp.services` is where we aggregate license usage and as such we *strongly recommend* including this in egress allow lists to ensure our partnership with your organization can be right-sized for your needs going forward.
        - Algolia - The Terraform Registry uses Algolia to index the current resources in the registry.
    - Additional outbound targets for VCS/SAML and so on depending on the use case.
    - Public cloud cost estimation APIs as necessary.

- This recommended architecture requires Terraform Enterprise `v202309-1` or later. Review the main public documentation if you have not already.
- The HVD Modules all include example code which you can use to deploy Terraform Enterprise on RHEL (Podman) or Ubuntu (Docker) onto new VM instances, which the module configures with the container runtime appropriate for the operating system chosen.

For production VM deployments we recommend the `external` or `active-active` operational modes together with the HashiCorp-maintained modules below. If you need a `disk` mode instance for development or testing, build an automated deployment that lets you destroy and recreate instances iteratively — we recommend Packer for this.

In some regulated environments, outbound access has limits or is totally inaccessible from server environments. If you need to run Terraform Enterprise fully air-gapped from the Internet, download provider and Terraform binary versions manually and host them in the Terraform Enterprise registry when these release, to offer them to your users.

To allow Terraform Enterprise access to the public registry, but prevent your user base from accessing community content, we recommend using Sentinel or OPA as part of platform development to limit which providers sign off for use.

5.2 Terraform modules for installation

To install Terraform Enterprise on VMs using a container runtime, we recommend you use the Terraform Enterprise HVD Modules and the Terraform command-line tool. Modules for deployment on VMs with Docker/Podman are available in the Terraform Registry for the following cloud providers.

Cloud Provider	Registry Link
AWS	Terraform Registry
Azure	Terraform Registry
GCP	Terraform Registry

HashiCorp Professional Services and HashiCorp partners use the installation code we provide to set up best practice Terraform Enterprise instances. We strongly recommend engaging partners or HashiCorp Professional Services to accelerate the scaling out of your project.

If you install Terraform Enterprise yourself, we recommend that you follow these high-level steps:

1. Import the provided Terraform Enterprise modules into your VCS repository.
2. Establish where to store the Terraform state for the deployment. HashiCorp recommends that, for simplicity, you store the state in cloud-based object store service (S3, Blob Storage, GCS, and so on) for which HVD Modules all contain a `backend.tf` file which you can use to configure the remote state storage location as this is more secure than using the local backend. Free access in HCP Terraform for this can also work.
3. Select a machine where the Terraform code runs. This machine needs to have the Terraform command-line tool available.
4. Ensure that you instantiate cloud credentials in the shell used on the machine for Terraform execution.

> **Note**
>
> Terraform state contains sensitive information and needs protection. We do not recommend that you store the state file for a Terraform Enterprise deployment in VCS or any unprotected location. It is the only state you need to separately secure, as all other state generated by your organization receives protection by Terraform Enterprise.

5.3 Terraform Enterprise license

Before starting the Terraform Enterprise installation, make sure you have a valid Terraform Enterprise license. If you do not, first reach out to your HashiCorp account team to request the license file. When you receive it, save it as `terraform.hclic`, and protect it as a company asset.

Ensure that the license file contains a single line with no new line character. Run the following command and ensure the output is `0`.

```
$ wc -l terraform.hclic
       0 terraform.hclic
```

5.4 Process overview

The layout of the HVD Module GitHub repositories follows a standard structure exhibiting these features:

- The Terraform code separates into logical `.tf` files at the top level of the repository, without the need for submodules and without calls to external child modules. This keeps the codebase whole and straightforward to understand.
- The main repository `README.md` file contains the primary instructions which you must read through and then follow to deploy Terraform Enterprise.
- Subdirectories in the respective repository include:
  - `docs` - Auxiliary documentation for the module.
  - `examples` - This directory contains example code to deploy Terraform Enterprise with several configurable options like operating system, container runtime, log forwarding, and so on.
  - `templates` - Contains HCL templates used by the module as needed.
  - `tests` - Contains deployment configuration used to test the module contents.

To deploy Terraform Enterprise using the provided modules, you need to:

1. Select your cloud provider below, and then follow the link to the respective public Terraform Registry entry.
2. In the Registry, review the contents, then click in the `Source Code` link in the page header to point your browser to the GitHub repository.
3. Read the GitHub repository for the respective Terraform module in its entirety. Not doing so may result in a failed deployment.
4. Follow the repository `README.md` file step by step, ensuring you have all prerequisites in place *before* starting the deployment; these may take some time to arrange in your environment and require accounting for in project planning.
5. Ensure you have the TLS certificate and private key for your Terraform Enterprise installation. The DNS SAN in the certificate must include the FQDN you use in DNS for the service (which

resolves to the NLB). We also expect you have a standard organizational CA bundle and process for generating these which we recommend using. We do not recommend self-signed certificates in any environment. Inspect your certificate with this command.

```
openssl x509 -noout -text -in cert.pem
```

1. The `README.md` directs you to complete the configuration and deploy Terraform Enterprise using the `terraform init`, `terraform plan` and `terraform apply` commands. When using the example `terraform.tfvars.example` file, remember to remove angled brackets from the values of key-value pairs in the resulting `terraform.tfvars` file.
2. Once the Terraform run completes, ensure to login to the VM and tail the automated application setup logs, watching for errors. Then return to the `README.md` to complete the task.
3. As soon as the software installs, you have *sixty minutes* to access the Initial Admin Console Token (IACT) which the system generates during the installation process. As part of your preparation, read the HashiCorp public documentation for using the IACT (Initial Admin Console Token) to create the first administrator user. This is also linked at the end of the main HVD Module `README.md` as next steps.

5.5 Container-specific guidance

More detailed guidance on the deployment of Terraform Enterprise on Docker appears in this section.

5.5.1 General guidance

Ensure that project planning allows time and resource for scale testing as close to the degree of scale that is eventually expected in production. We recommend working with your earlier adopter teams to understand the expected scale and to ensure that you size the machines appropriately during development and testing.

Ensure that observability tooling is also in place *before* load testing so that you can fully understand CPU, RAM and I/O constraints in your specific context and in terms of connectivity to external services.

5.5.2 CPU

At high concurrency, HCP Terraform agent workload may pressure network throughput and is sensitive to the over-allocation of CPU. Evaluation of memory-optimized instances shows they are unable to provide sufficient CPU, resulting in possible HCP Terraform agent workspace run failures.

5.5.3 RAM

Memory sizing is the most workload-dependent and RAM usage depends on the Terraform configuration executed. To size conservatively, start with the system defaults and test thoroughly, preferably using representative workloads, and increase limits as necessary.

The default HCP Terraform agent run pipeline configures a container resource request for every agent at 2 GB. If the instance is not appropriately sized for this reservation, physical memory over-allocation causes run failure. You can adjust this using the `TFE_CAPACITY_MEMORY` environment variable directive. The conservative approach is to size on the limit so you can tune this down if cost efficiency is a priority. If it needs to be uniformly increased, test it exhaustively. If only some workspaces require increases on this limit, consider running a agent pool for those workspaces, where dedicated hardware can run agents with higher resource availability as needed.

Determine maximum HCP Terraform agent RAM requirement and production system overhead

The `TFE_CAPACITY_CONCURRENCY` variable manages the number of concurrent workspace runs, and at the default, RAM requirement equates to `TFE_CAPACITY_CONCURRENCY * 2GB`. In the nominal example preceding, with a requirement of 30 agents (an agent container runs a workspace run), the maximum RAM requirement would be thus 60 GB for workspace runs.

For right-sizing of the instance, at least ten percent overhead is prudent, making the total RAM requirement 66 GB. Note that this calculation does not include OS requirements or for agents you need to run in discrete network segments outside of the instance, which you would size separately based on specific graph calculation requirements.

5.6 Platform-specific guidance

Select your cloud provider below for further guidance and corresponding machine size choice.

For cluster calculation below:

- One node in each of three AZs.
- Nominal Linux operating system RAM requirement of **2048MiB**.
- Terraform Enterprise requiring **4096MiB**.
- Cluster calculations in MiB based on the `TFE_CAPACITY_MEMORY` system default of **2048MiB**.
- Part GB calculations round down.

5.6.1 Resource sizing

The following table maps the baseline component sizing to recommended instance types and managed-service SKUs for each cloud provider.

Component	AWS	Azure	GCP
Disk	EBS gp3 (3000 IOPS)	Premium SSD (5000 IOPS)	Balanced Persistent SSD (10000 IOPS)
Machine (default)	`m7i.2xlarge` (8 vCPU, 30517MiB)	`Standard_D8s_v4` (8 vCPU, 30517MiB)	`n2-standard-8` (8 vCPU, 30517MiB)
Machine (scaled)	`m7i.4xlarge` (16 vCPU, 61035MiB)	`Standard_D16s_v4` (16 vCPU, 61035MiB)	`n2-standard-16` (16 vCPU, 61035MiB)
Database	`db.r6i.xlarge`	`GP_Standard_D4ds_v4`	`db-custom-4-16384`
Cache	`cache.m5.large`	`Premium`, a family of `P` and a capacity of 1	`STANDARD_HA`

For ideal CPU sizing across all providers:

- Avoid instances with burstable CPU or network characteristics, for example:
  - AWS: T type

  - Azure: B type
  - GCP: e2-, f1-, g1- types

- Choose the latest generation general purpose instance type x86-64 hosts.
- Use CPU/RAM ratio of 1:4 or higher.
- Do not use memory-optimized instances.

5.6.2 Cluster calculations

The default machine type (8 vCPU, 30517MiB) provides:

- 30517MiB - 2048MiB - 4096MiB = 24373MiB RAM maximum spare for workspace runs (25 GB).
- Maximum `TFE_CAPACITY_CONCURRENCY` of 24373MiB / 2048MiB = **11**.

The scaled machine type (16 vCPU, 61035MiB) provides:

- 61035MiB - 2048MiB - 4096MiB = 54891MiB RAM maximum spare for workspace runs (57 GB).
- Maximum `TFE_CAPACITY_CONCURRENCY` of 54891MiB / 2048MiB = **26**.

5.7 Observability

Enable the general Terraform Enterprise metrics, logs, and agent telemetry described in Observability setup; that setup is common to all platforms. On a VM deployment, forward Terraform Enterprise logs and metrics to the observability service native to your cloud:

- **AWS**: Amazon CloudWatch, with Amazon S3 for log archival.
- **Azure**: Azure Monitor and Azure Log Analytics, or Azure Blob Storage for log archival.
- **GCP**: Google Cloud Logging.

Put observability tooling in place *before* you load-test, so that you can understand CPU, RAM, and I/O constraints — and connectivity to the external services — in your specific context. Ongoing, day-2 observability and alerting is covered in the Administration Guide.

6 Deployment (managed Kubernetes)

This page provides guidance on how to deploy Terraform Enterprise on EKS, AKS and GKE using Terraform modules written and managed by HashiCorp (HVD Modules). The modules work in conjunction with the Terraform command-line tool, are available in the HashiCorp Terraform Registry and are the best practice method for deploying Terraform Enterprise on the respective managed Kubernetes services. See below for more on this.

Note

This topic requires a good understanding of Terraform Enterprise and managed Kubernetes services. Read the architecture section before this one.

6.1 Architectural summary

- Deployment of Terraform Enterprise using managed Kubernetes requires the `active-active` operational mode where a Redis instance operates separately, even if only one application container operates.
- Deploy one or more Terraform Enterprise pod(s) onto a managed Kubernetes instance that autoscales across availability zones (AZs) and which reduces the overhead in managing those services including patching and upgrading.
- Use managed versions of object storage, PostgreSQL database and clustered Redis cache (as offered by the public cloud of choice, but specifically not Redis Cluster which is not supported), to ensure replicas distribute in different AZs.
- Use a layer 4 load balancer to ingress traffic to the Terraform Enterprise instances. This is because:
    - Certificates need to be on the compute nodes for Terraform Enterprise to work.
    - It is more secure to terminate the TLS connection on Terraform Enterprise rather than outside it and have to re-encrypt traffic from the inside of the load balancer requiring an additional certificate.
    - It is more straight forward to manage than a layer 7 load balancer.
- By using three AZs (one Terraform Enterprise pod in each AZ), the system has an n-2 failure profile, surviving failure of two AZs. However, if the entire region is unavailable, then Terraform Enterprise experiences an outage. The application architecture is single-region.

- We do not recommend exposing Terraform Enterprise to the public Internet. Users must be on the company network to be able to access the Terraform Enterprise API/UI. However, we recommend allowing Terraform Enterprise to access certain addresses on the Internet:
  - The HashiCorp container registry - where HashiCorp provides the Terraform Enterprise container
  - HashiCorp service APIs (all owned and operated by HashiCorp except Algolia):
    - `registry.terraform.io` houses the public Terraform module registry which enterprise customers want to avoid letting users have unfettered access to (see below). However, this is where official providers index.
    - `releases.hashicorp.com` is where HashiCorp host Terraform binary releases. We recommend users stay within two minor releases of current to access the latest security updates, and new features.
    - `reporting.hashicorp.services` is where we aggregate license usage and as such we *strongly recommend* including this in egress allow lists to ensure our partnership with your organization can be right-sized for your needs going forward.
    - Algolia - The Terraform Registry uses Algolia to index the current resources in the registry.
  - Additional outbound targets for VCS and SAML depending on the use case.
  - Public cloud cost estimation APIs as necessary.
- Terraform Enterprise `v202309-1` or later works with Kubernetes deployments. Review the main public documentation for this architecture if you have not already.
- The HVD Modules all include example code which you can use to deploy Terraform Enterprise on an existing Kubernetes cluster, or to deploy a new cluster and then deploy the application onto it. To make the most efficient use of computing resources, we recommend deploying Terraform Enterprise onto an existing cluster.
- Assess supported versions of Kubernetes from the latest Terraform Enterprise Releases document.
- Remember that when selecting machine types in this guide, Terraform Enterprise supports x86-64 on all versions, but deployment on ARM architecture requires version v1.0.0 or later.
- The Helm chart that deploys Terraform Enterprise has versions in this repository and you must read and understand it as part of your preparation for deployment.

In some regulated environments, outbound access to the Internet limits or totally restricts server environments. If you need to run Terraform Enterprise in a fully air-gapped mode, you must manu-

ally download and host provider and Terraform binary versions in the Terraform Enterprise registry when these release, to offer them to your users.

To allow Terraform Enterprise access to the public registry, but prevent your user base from accessing community content, we recommend using Sentinel or OPA as part of platform development to limit which providers you sign off for use.

If you are planning a scaled deployment, ensure your project management team allocates significant resources to engineer a suitable policy-as-code SDLC deployment. It must be ready for UAT at the latest such that users adapt to restrictions *well before* the production deployment.

6.2 Terraform modules for installation

The primary route to the installation of Terraform Enterprise on Kubernetes is through the use of HVD Modules which require the use of a Terraform command-line tool binary. These modules are available in the public HashiCorp Terraform Registry.

The installation code we provide supports HashiCorp Professional Services and HashiCorp partners who set up best practice Terraform Enterprise instances. We highly recommend leveraging partners or HashiCorp Professional Services to accelerate the scaling out of your project.

If you install Terraform Enterprise yourself, we recommend that you follow these high-level steps:

1. Import the provided Terraform Enterprise modules into your VCS repository.
2. Establish where to store the Terraform state for the deployment. HashiCorp recommends that you store state in HCP Terraform (free access is available or contact your HashiCorp account team for an entitlement for state storage just for Terraform Enterprise installs). If access to HCP Terraform is not possible, we recommend using a secure, cloud-based object store service (S3, Blob Storage, GCS and so on) instead.
3. Select a machine where you execute the Terraform code. This machine needs the Terraform command-line tool available. Use the latest binary version.
4. Ensure that cloud credentials are available on the machine for Terraform execution.

Note

Terraform state contains sensitive information and needs protection. We do not recommend that you store the state file for a Terraform Enterprise deployment in VCS or any unprotected location. It is the only state you need to separately secure, as Terraform Enterprise protects all other state generated by your organization.

6.3 Process overview

The layout of the HVD Module GitHub repositories follows a standard structure exhibiting these features:

- The Terraform code separates into logical `.tf` files at the top level of the repository, without the need for submodules and without calls to external child modules. This keeps the codebase whole and straightforward to understand.
- The main repository `README.md` file contains the primary instructions which you must read through *first* and then follow to deploy Terraform Enterprise.
- Subdirectories in the respective repository include:
  - `docs` - Auxiliary documentation for the module.
  - `examples` - This directory contains more than one example use case, each of which pertain to a root module which, when configured and run, uses the module to deploy Terraform Enterprise. Expect to run at least the initial development deployment from one of these subdirectories.
  - `templates` - Contains HCL templates used by the module as needed.

To deploy Terraform Enterprise using the provided modules, you need to:

1. Select your managed Kubernetes service below, and then follow the link to the respective public Terraform Registry entry.
2. In the Registry, review the contents, then click the `Source Code` link in the page header to point your browser to the GitHub repository.
3. Read the GitHub repository for the respective Terraform module *in its entirety*. Not doing so may result in a failed deployment. Do not run code you do not understand.
4. Follow the repository `README.md` file step by step, ensuring you have all prerequisites in place *before* starting the deployment; these may take some time to arrange in your environment and you must account for them in project planning.

5. Ensure you have the TLS certificate and private key for your Terraform Enterprise installation. The DNS SAN in the certificate must include the FQDN you use in DNS for the service (which resolves to the Kubernetes NLB). We also expect you have a standard organizational CA bundle and process for generating these which we recommend using. We do not recommend self-signed certificates, especially in production environments. Inspect your certificate with this command.

```
openssl x509 -noout -text -in cert.pem
```

1. The `README.md` directs you to complete the configuration and deploy Terraform Enterprise using the `terraform init`, `terraform plan` and `terraform apply` commands. If using the example `terraform.tfvars.example` file, remember to remove angled brackets from the resulting `terraform.tfvars` file.
2. Once the Terraform deployment completes, insert the relevant secrets into the cluster. Do this in a way which does not result in secrets storing anywhere else (for example `.bash_history`, VCS and so on). The HVD Modules have a document about Kubernetes secrets management detailing recommendations for different scenarios and lists the secrets you need to use. If your organization has a strategic Kubernetes secrets management approach which avoids writing plain text secrets, we recommend using it. Otherwise, use `read -sp` as per the following command example for each secret in scope to securely instantiate environment variables into your shell and then use these with `kubectl` to ingress them into Kubernetes as the HVD Module instructs.

```
read -sp 'TFE_ENCRYPTION_PASSWORD> ' TFE_ENCRYPTION_PASSWORD && export
    TFE_ENCRYPTION_PASSWORD
```

1. The modules contain a `locals_helm_overrides.tf` convenience function which generates a file called `helm/module_generated_helm_overrides.yaml` - this overrides certain default values in the Helm chart for Terraform Enterprise.
2. The instructions then guide you as to the use of Helm to deploy the Terraform Enterprise container to your Kubernetes cluster.
3. Follow the last link in the README to the information on the HashiCorp public documentation for using the IACT (Initial Admin Console Token) to complete the setup of Terraform Enterprise *within 60 minutes of deployment*.

6.4 Kubernetes-specific guidance

This section provides more detailed guidance on the deployment of Terraform Enterprise on Kubernetes.

6.4.1 General guidance

- Separate Terraform Enterprise pods and HCP Terraform agent worker pods, because even under load, Terraform Enterprise pod resource consumption is more consistent than HCP Terraform agent workload requirements which are necessarily inconsistent and demanding on CPU and network I/O. Node separation for Terraform Enterprise pods and HCP Terraform agents is preferable when operating at scale as a result.
- Use the HCP Terraform Operator instead of the internal Terraform Kubernetes driver run pipeline (see below). The internal run pipeline provisions all agents on demand, creating a much more inconsistent and spiky workload during peak demand.
- Three pods for Terraform Enterprise are sufficient as this is more for availability than performance. HCP Terraform agent cluster node capacity has the greatest impact on run success at scale.
- Ensure that project planning allows time and resource for scale testing as close to the degree of scale that is eventually expected in production. We recommend working with your adopter teams to understand the expected scale and to ensure appropriate cluster sizing during development and testing.
- Ensure that observability tooling is also in place *before* load testing so that you fully understand CPU, RAM and I/O constraints in your specific context and in terms of connectivity to external services.

6.4.2 Use of the internal run pipeline versus HCP Terraform Operator

The HCP Terraform Operator is more efficient at spreading agent demand by establishing minimum numbers of replicas and preventing thundering herd issues that can occur with the internal Terraform Kubernetes driver run pipeline where all agents come online at once. However, smaller deployments may find the internal run pipeline sufficient.

For customers going beyond the default concurrency per Terraform Enterprise pod, it is highly preferable to leverage the HCP Terraform Operator in all situations. The sizing guidelines remain

the same.

6.4.3 CPU

At high concurrency, HCP Terraform agent workload may pressure network throughput and is sensitive to the over-allocation of CPU. Memory-optimized instances are less performant and cannot provide sufficient CPU, resulting in possible HCP Terraform agent workspace run failures.

Do not use burstable instances with low baseline throughput limitations on CPU and network. For example, avoid T type AWS instances. Use the latest generation Intel or AMD instances where possible. See below for cloud-specific guidance on machine sizing.

More Terraform Enterprise pods does not equal improved performance and can reduce performance unless you give careful consideration to the impact of additional pods on the database I/O, and the concurrency multiplier (number of Terraform Enterprise pods * concurrency).

6.4.4 RAM

Memory sizing is the most workload-dependent and the Terraform configuration executed drives RAM usage. To size conservatively, start with the system defaults and test thoroughly, preferably using representative workloads, and increase limits as necessary.

The default HCP Terraform agent run pipeline configures a pod resource request for every agent at 2048MiB, so if the cluster is not appropriately sized for this reservation, physical memory over-allocation causes run failure. You can adjust this using the `agentWorkerPodTemplate` directive in the Helm `module_generated_helm_overrides.yaml` if you need to adjust the max concurrent workload memory usage. The conservative approach is to size on the limit so you can tune this down if cost efficiency is a priority.

The Terraform Enterprise internal run pipeline configures the Kubernetes driver to use a default resource request and limit of 2048MiB per agent. If running the defaults, you need to factor this into your cluster sizing (number of Terraform Enterprise pods * `TFE_CAPACITY_CONCURRENCY`). You must size HCP Terraform agent nodes to handle the configured run concurrency, as over-allocation of physical node memory results in consistent run failure. The default configuration assumes every agent uses a max of 2 GB at the same time. As an example, this could be possible with a large Terraform configuration at high concurrency. Typical scenarios to be aware of include a repetitive platform team workflow such as the use of landing zones at scale.

By default, the HCP Terraform operator configures no resource limits on the HCP Terraform agents. Ideally, set a limit on memory and configure a baseline resource request. This can help efficient node placement and becomes critical if using a cluster scaling technology such as Karpenter.

Determine maximum HCP Terraform agent RAM requirement and production system overhead

You set this resource capacity in your `module_generated_helm_overrides.yaml` file and it is thus configurable irrespective of whether you deploy agents using a pipeline or the HCP Terraform Operator for Kubernetes. The following configuration specifies the default allocated RAM resource request of 2 GB.

```
resources:
  limits:
    memory: 2048M
  requests:
    memory: 2048M
```

From this, the maximum RAM requirement for the agents is thus calculated as `number of agents * 2GB`. In the nominal example, with 30 agents, the maximum RAM requirement would be 60 GB for workspace runs.

For right-sizing of the cluster, at least ten percent overhead is prudent, making the total RAM requirement 66 GB. Note that this calculation does not include OS requirements or for agents that run in discrete network segments outside of the cluster, which you size separately based on specific graph calculation requirements.

Select your managed Kubernetes service below for further specific guidance on CPU sizing and corresponding machine size choice.

> **Note**
>
> These numbers assume, in a scaled environment, HCP Terraform agents run in a dedicated node group/node pool or cluster, as we recommend isolation from the Terraform Enterprise pods. If running in a shared environment, provide at least 4 GB memory per Terraform Enterprise pod to ensure you apply appropriate resource requests to Terraform Enterprise pods for some protection under load.

6.4.5 Network

Reduce egress network load by specifying a specific version tag for the HCP Terraform agent image - `tfc-agent:<tag>`. The use of `tfc-agent:latest` results in retrieving the image every time there is a workspace run and thus unnecessary network load. This becomes more significant when using the internal pipeline as you deploy all workers on demand.

The HVD Modules all deploy layer 4 load balancers. This is the highest throughput load balancer available. Ensure that you load the HCP Terraform agent docker image from a performant region local source where possible like ECR, rather than a public Internet-based source.

Do not use instances with burstable network characteristics.

6.4.6 Disk

The Terraform Enterprise pods and HCP Terraform agent workloads are not storage I/O bound in Kubernetes (I/O moves to the external services). During testing, baseline root partitions did not see any latency spikes on nodes under load and node storage did not become a factor at scale.

6.4.7 Machine sizing

Customers must understand the expected target scale for the project and ensure the instances which comprise the cluster are of sufficient capacity accordingly. While the architecture page in this HVD provides initial guidance on sizing, consider the calculations below.

Determine pod count

Use a nominal, initial expectation of three Terraform Enterprise pods for high availability, one deployed in each of three AZs in each of three replicas (total three pods across three AZs). Increase in sets of three so that a balance of pod availability maintains across the chosen region.

Determine HCP Terraform agent count requirements

Calculate how many HCP Terraform agents you need by using this formula:

Number of HCP Terraform agents = `TFE_CAPACITY_CONCURRENCY` * number of Terraform Enterprise pods

`TFE_CAPACITY_CONCURRENCY` defaults to 10, so with the initial pod count of three, the agent capacity expected is 30.

6.5 Platform-specific guidance

Select your managed Kubernetes service below for further guidance and corresponding machine size choice.

6.5.1 Deployment modules

Select your managed Kubernetes service for the official HVD module link.

EKS

The official Terraform Enterprise deployment module for Elastic Kubernetes Service is available at this entry in Terraform Registry.

AKS

The official Terraform Enterprise deployment module for Azure Kubernetes Service is available at this entry in Terraform RegistryNot every machine size is available in every region. You may find that machine sizes are available for compute in a region, but unavailable for AKS in the same region. We recommend checking the availability of the machine sizes in the region you intend to deploy to by using the Azure console. Start the deployment of an AKS instance just to see which machine sizes it offers, making a note of the correct machine size for your deployment and then cancel the deployment.Use the following command to identify the version of Kubernetes available for AKS in the region you intend to deploy to, replacing <region> with your region (for example `northeurope` or `eastus`). Cross reference this with the version of Kubernetes supported by Terraform Enterprise to ensure you deploy a version of Kubernetes which Terraform Enterprise supports.`shell az aks get-versions --location <region> --output table`

GKE

The official Terraform Enterprise deployment module for Google Kubernetes Engine is available at this entry in Terraform Registry

6.5.2 Resource sizing

The following table maps the baseline component sizing to recommended instance types for each managed Kubernetes service.

Component	EKS	AKS	GKE
Disk	EBS gp3	Premium SSD Managed Disks	Persistent SSD Disks

Component	EKS	AKS	GKE
Machine (3-node cluster)	`m7i.2xlarge` (8 vCPU, 32 GB)	`Standard_D8s_v5` (8 vCPU, 32 GB)	`n2-standard-8` (8 vCPU, 32 GB)
Machine (5-node cluster)	`m7i.xlarge` (4 vCPU, 16 GB)	`Standard_D4s_v5` (4 vCPU, 16 GB)	`n2-standard-4` (4 vCPU, 16 GB)

Example approximate minimum Kubernetes cluster sizings for HCP Terraform agents only, with three Terraform Enterprise pods at system defaults are as follows.

- 3-node cluster: 96 GB total memory, 64 GB (n-1)
- 5-node cluster: 80 GB total memory, 64 GB (n-1)

For ideal CPU sizing across all providers:

- Choose the latest generation general purpose instance type x86-64 hosts.
- Use CPU/RAM ratio of 1:4 or higher.
- Do not use memory-optimized instances.

6.6 Observability

Enable the general Terraform Enterprise metrics, logs, and agent telemetry described in Observability setup; that setup is common to all platforms. On a managed Kubernetes deployment, forward Terraform Enterprise and HCP Terraform agent logs and metrics to the observability service native to your cloud — Amazon CloudWatch on EKS, Azure Monitor and Azure Log Analytics on AKS, and Google Cloud Logging on GKE — and scrape the Terraform Enterprise metrics endpoint with your cluster's metrics stack, such as Prometheus. The HCP Terraform agent exposes telemetry over OpenTelemetry, so route it through an OpenTelemetry collector to your monitoring system.

Put observability tooling in place *before* you load-test, so that you can understand CPU, RAM, and I/O constraints — and connectivity to the external services — in your specific context. Ongoing, day-2 observability and alerting is covered in the Administration Guide.

6.7 Troubleshooting

The following are common issues that may arise during the deployment of Terraform Enterprise on Kubernetes.

6.7.1 ImagePullBackOff

This error occurs when the Kubernetes cluster is unable to pull the Terraform Enterprise container image from the HashiCorp container registry. This can be due to a number of reasons, including:

- The Kubernetes cluster does not have the necessary permissions to pull the image.
- The image is not available in the HashiCorp container registry. Check the version of Terraform Enterprise you have specified in the `locals_helm_overrides.tf` file.
- The image pull secret is not correctly configured in the Kubernetes cluster. Ensure that you have processed the license file HashiCorp have issued you. The license file must not have a new line in it if you intend to run the equivalent of `cat tfe.hclic | base64` to generate the base64 encoded license string while populating Kubernetes secrets.

6.7.2 Crash loop back off

This error occurs when the Terraform Enterprise container is unable to start correctly. Again, this can be due to a number of reasons. To diagnose the problem, open two terminal windows, and in one, run:

```
while true
do
  sleep 1
  kubectl exec -n tfe -ti $(kubectl get pods -n tfe \
  | tail -1 \
  | awk '{print $1}') -- tail -n 100 -f /var/log/terraform-enterprise/terraform-
      enterprise.log
done
```

and in the other, run your `helm install` command. This means that the moment the container starts up, you start to see the initial output to the terminal running the **`while`** loop. This at least provides the error message from the start up of Terraform Enterprise that helps diagnose the problem, or that you can pass to HashiCorp Support when raising a support ticket.

7 Deployment (manual)

Our Terraform-based Terraform Enterprise deployment modules focus on public cloud deployments because public CSPs offer a predictable set of services. Even though the overwhelming majority of our private cloud customers use VMware, there is still a bespoke approach to deployment, instead using a wide variety of additional configuration management technologies and deployment approaches to manage software installation. As such, an opinionated private cloud deployment Terraform module would be unlikely to be sufficiently useful for all private cloud customers.

This section thus provides recommendations and best practices for scaled, enterprise private cloud customers who need to apply their organisation-standard tooling to the deployment of Terraform Enterprise. If you are deploying Terraform Enterprise to a public cloud, refer to the Terraform-based Terraform Enterprise installation section for instructions.

7.1 Deployment topology

As listed in the architecture section, HashiCorp recommends the Active/Active deployment topology for customers seeking to guarantee high availability, resilience and scale. As an automated deployment is a requirement of the `active-active` operational mode, engineering to provide this must be a headline planning item for the project manager.

7.2 Component considerations

This section discusses a series of considerations specific to `active-active` private cloud installations irrespective of whether you deploy Terraform Enterprise using Docker or Kubernetes.

7.2.1 Redis

As a result of there being multiple concurrent instance nodes of Terraform Enterprise, your organization must consider the deployment of a Redis instance in your private datacenter as this is a hard requirement for `active-active`. We recommend customers for whom this proves impossible to consider a public cloud deployment, or HCP Terraform if SaaS is a viable option for your organization.

To avoid arranging product ownership and ongoing maintenance of Redis, the only self-managed alternative is to use the `external` operational mode instead, but this means there being only one Terraform Enterprise container in the instance. In certain circumstances, an orchestrator, such as Kubernetes, can restart the container fast enough for your needs, and your RTO dictates this as a viable possibility, then you can avoid a separate Redis instance. In our experience however, your business requires higher availability and lower RTO over time, so we recommend, forward planning for Redis support on-premise in due course.

7.2.2 PostgreSQL

Many customers use alternatives to PostgreSQL as their strategic database solutions. Terraform Enterprise depends specifically on PostgreSQL, which means a private cloud deployment requires that your organization has a pattern for the deployment and production management of PostgreSQL at the versions listed on our website.

Also be aware that there are specific schema requirements in the preceding link, so we advise liaising in the first instance with your DBA team so that all parties are aware of the requirements at the outset.

7.2.3 S3-compatibility

In addition to Redis and PostgreSQL, Terraform Enterprise requires S3-compatible storage to operate. In a private datacenter scenario, this means adopting a third-party technology to present an S3-compatible API to Terraform Enterprise.

Traditionally, HashiCorp has pointed customers who do not already have a solution in place to the most-used examples, however it is not for us to comment on the suitability of one solution for your business over another.

From working with our customers, we see significant success with private cloud deployments of Terraform Enterprise using either Dell ECS or MinIO as the object storage tier. See here for MinIO setup guide for Terraform Enterprise. If your organization has a pattern for on-premise S3-compatible object storage, our recommendation is to use that.

7.3 Component sizing

Terraform Enterprise and its external components follow the baseline component sizing defined in the detailed design. In a manual private-cloud deployment you provision the application hosts, PostgreSQL, and Redis yourself, so size each of them to meet or exceed that baseline. The notes below are specific to a self-managed deployment.

7.3.1 Operating system

We recommend deploying Terraform Enterprise on one of the following operating systems:

- Red Hat Enterprise Linux
- Ubuntu LTS

7.3.2 Compute

Each Terraform Enterprise application instance runs as a container. Configure CPU and memory identically across instances, and remember the VM must be sized higher than the Terraform Enterprise application limits to allow for the operating system and any adjacent containers. For ongoing right-sizing and scale-out, see the Administration Guide.

The disk backing each host must sustain a minimum of 3000 IOPS, and Terraform Enterprise requires at least 40 GB of available space; for Docker deployments this space must be available to `/var/lib/docker`.

Object store consumption depends on two things:

1. The number of workspace runs — each run generates objects in the object store irrespective of whether the run succeeds or fails.
2. The size of configuration directories submitted to the API during Terraform runs. Terraform aggregates the configuration directory and sends it to the API, which commits it to encrypted object storage through an internal process called archivist. Limit your configuration repositories to only the files a run needs — a 50 MB `.tgz` file in a configuration directory needlessly adds 50 MB to your object store and wastes hosting cost.

7.3.3 PostgreSQL and Redis

Provision PostgreSQL and Redis to the baseline component sizing, at the versions documented in the detailed design. Because these are self-managed in a private-cloud deployment, confirm your organization has an established pattern for deploying and operating them at the required versions before you begin.

7.4 Observability

Enable the general Terraform Enterprise metrics, logs, and agent telemetry described in Observability setup; that setup is common to all platforms. The guidance below is specific to a manual private-cloud deployment, where you own the underlying hosts and the self-managed PostgreSQL and Redis instances.

Monitor CPU, memory, available disk space, and disk I/O on the application hosts and on the self-managed PostgreSQL and Redis servers using your standard organizational machine telemetry solution. Create alerts for threshold breaches at 50% and 70% utilization; if any parameter exceeds 70% consistently, increase the resource to maintain performance. Report telemetry on an ongoing basis so the team that owns the instance is aware of resource consumption.

Ongoing, day-2 observability and alerting is covered in the Administration Guide.

7.5 Networking requirements

Terraform Enterprise has a number of ingress and egress requirements. Please refer to the network requirements page for the latest information. If a corporate proxy service is filtering outbound Internet traffic, then add the destinations detailed in the preceding link to your allow-list.

> **Note**
>
> If your organization is not able to provide access to the required domains, Terraform Enterprise can work in an air-gapped mode. Please refer to the documentation here.

Use a layer 4 load balancer with Terraform Enterprise.

7.6 Installation process - Docker

Write an automated Terraform Enterprise installation process which conforms to your organizational requirements. We use Bash for illustration. Write required tasks into automation so that each machine starts up and installs the software the same way.

We recommend reading through the below prior to running the steps manually as part of your planning for automation.

7.6.1 Prepare requirements

The following list summarizes the infrastructure resources required prior to the deployment of Terraform Enterprise.

1. VMs for installation of Terraform Enterprise
2. PostgreSQL database
3. Redis cache
4. S3-compatible object storage
5. Terraform Enterprise license file (if you already have a license file but are on a Replicated-backed instance, you need a replacement file, but the contract is the same)
6. TLS certificate bundle
7. Load balancer

7.6.2 Tasks

As root, perform the equivalent of these commands:

```
sudo mkdir -p /opt/tfe/certs
cd !$
```

Put the PEM-encoded server certificate, private key and CA bundle files in this directory. For the sake of this installation, we assume the following filenames `cert.pem`, `key.pem` and `bundle.pem`.

```
tfe
`-- certs
    |-- bundle.pem
    |-- cert.pem
    `-- key.pem
```

> **Warning**
>
> Terraform Enterprise cannot use passphrase-protected private keys.

7.6.3 Create a Docker Compose configuration

Write a Docker configuration with the consolidated information about your `active-active` environment. Base this `compose.yml`file on this one. Note that you must be using on-premise S3-compatible object storage, so use the `TFE_OBJECT_STORAGE_*` variables listed in the link as you would were you to deploy Terraform Enterprise on AWS.

> **Warning**
>
> The `compose.yml` contains sensitive information that needs protection. Please treat this file as you would any other company secret.

In the configuration properties, we want to highlight the following:

- Treat the `TFE_ENCRYPTION_PASSWORD` value as a sensitive secret. We recommend using a password generation utility/process to follow corporate policies. Terraform uses this password to decrypt the Vault token stored in the database; Terraform Enterprise uses the Vault token to do all subsequent encryption/decryption processing.
- Terraform Enterprise uses an embedded instance of HashiCorp Vault for data security operations. When identifying the `TFE_VAULT_CLUSTER_ADDRESS` for the Vault instance, point to the private IP of the VM. The internal instrumentation of Vault performs a discovery to find peer instances and add to a private cluster so the IP addresses of all nodes in the cluster need to be accessible to all the other nodes.
- The `TFE_DATABASE_HOST` points to your PostgreSQL database cluster endpoint name.
- The `TFE_REDIS_HOST` points to the endpoint of the primary node in the Redis Replication Group.
- The `TFE_LICENSE` entry requires a raw string from your Terraform Enterprise license file. The license string is the `data.attributes.signed_string` object.

This is not an exhaustive list of configuration options. Please refer to the configuration reference for documentation about all the configuration options.

Write the `compose.yml` to a location where your nascent application machines can access it - each of them needs to use it for the installation.

7.6.4 Configure systemd

We recommend using the following `systemd` manifest to run `docker compose` on system start, so that if the machine reboots, Terraform Enterprise automatically starts. Create `/etc/systemd/system/terraform-enterprise.service` containing

```
[Unit]
Description=Terraform Enterprise Service
Requires=docker.service
After=docker.service network.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/etc/terraform-enterprise
ExecStart=/usr/local/bin/docker compose up -d
ExecStop=/usr/local/bin/docker compose down
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
```

then run

```
systemctl enable --now terraform-enterprise
```

7.6.5 Download the Terraform Enterprise image

First, login to the Terraform Enterprise container image registry using

```
cat <PATH_TO_HASHICORP_LICENSE_FILE> \
| docker login --username terraform images.releases.hashicorp.com --password-stdin
```

If you do not receive the output `Login Succeeded` using the provided license, contact your HashiCorp representative for support.

Next, pull the container image. Use the form `docker pull images.releases.hashicorp.com/hashicorp/terraform-enterprise:<vYYYYMM-#>` where `<vYYYYMM-#>` is the release required for example:

```
docker pull images.releases.hashicorp.com/hashicorp/terraform-enterprise:v202309-1
```

Check the releases page for the suitable version. We recommend using last month's latest release - if it is October, get `v202309-x` where `x` is the highest listed integer.

If necessary, you can pull the image and push it to your private Docker registry. When placing the image in a private Docker registry ensure to re-tag the image to support your hierarchical organization standards.

7.6.6 Run Docker Compose

Place the `compose.yml` in your working directory. When done, the directory structure must look like this:

```
tfe
|-- certs
|   |-- bundle.pem
|   |-- cert.pem
|   `-- key.pem
`-- compose.yml
```

Each node then needs to run

```
sudo docker compose up --detach
```

Messages indicating the progress of the running process result. Follow logs using this in a separate shell:

```
docker compose logs --follow
```

7.6.7 Run a health check

Terraform Enterprise provides a `/_health_check` endpoint on the instance. Expect a `200 OK` response from a healthy system.

The `/_health_check` endpoint operates in 2 modes:

- Full check with `/_health_check?full=1`
- Minimal check with `/_health_check`

With a full check, the service attempts to verify the status of internal components and PostgreSQL, in contrast to a minimal check which returns `200 OK` automatically after a successful full check. To run a health check, use:

```
sudo docker exec terraform-enterprise curl http://127.0.0.1:9292/_health_check?full=1
```

The command returns a JSON encoded message as follows:

```
{"postgres":"UP","redis":"UP","vault":"UP"}
```

Use the following or equivalent in your code to pause the deployment process until all Terraform Enterprise processes are healthy.

```
while ! docker compose exec terraform-enterprise curl http://127.0.0.1:9292/
    _health_check?full=1;
do
  echo "Waiting for Terraform Enterprise to be ready"
  sleep 30
done
```

The default behavior of this endpoint is to perform a full check during startup of the instance, and minimal checks after Terraform Enterprise is active and running. The Terraform Enterprise application supports a practical `tfe-health-check-status` command to provide full visibility. You can run the command in the VM as follows:

```
sudo docker exec terraform-enterprise tfe-health-check-status
```

When started, a healthy Terraform Enterprise instance returns messages as follows:

```
checking: Archivist Health Check...
|  checks that Archivist is up and healthy
|- ✓ PASS

checking: Terraform Enterprise Health Check...
|  checks that Terraform Enterprise is up and can communicate with Redis and Postgres
|- ✓ PASS

checking: Terraform Enterprise Vault Health Check...
|  checks that Terraform Enterprise can connect to Vault and is able to encrypt and
    decrypt tokens
|- ✓ PASS

checking: Fluent Bit Health Check...
|  checks that the configure Fluent Bit server is healthy
|- SKIPPED

checking: RabbitMQ Health Check...
|  checks that RabbitMQ can be connected to and that we can send and consume messages
|- SKIPPED

checking: Vault Server Health Check...
|  checks that the configured Vault Server is healthy
|- ✓ PASS

  All checks passed.
```

When forcing a full check, take extra caution as every call makes requests to all external services, increasing system load and latency. This is not a concern during the initial installation stages as there are no in-flight operations expected.

7.6.8 Retrieve your initial administrator creation token

When you have deployed all Terraform Enterprise application instances, you can retrieve the initial administrator creation token. Choose one of your EC2 Instances and run the following command in a terminal session:

```
docker compose exec terraform-enterprise retrieve-iact
```

The command returns your initial administrator creation token. Protect this sensitive token as part of the handover process to the operating teams. The form is as follows.

5db5c3ae5be1599804bc614ec976e1a64ad1cadf76fec518d61ed9d04ef565eb

To view the startup web page, open a web browser and navigate to the Terraform Enterprise URL.

The syntax for the URL endpoint is: `$URL/admin/account/new?token=$TOKEN`

For example:

```
https://alb_entry_point.your_domain.com/admin/account/new?
token=5db5c3ae5be1599804bc614ec976e1a64ad1cadf76fec518d61ed9d04ef565eb
```

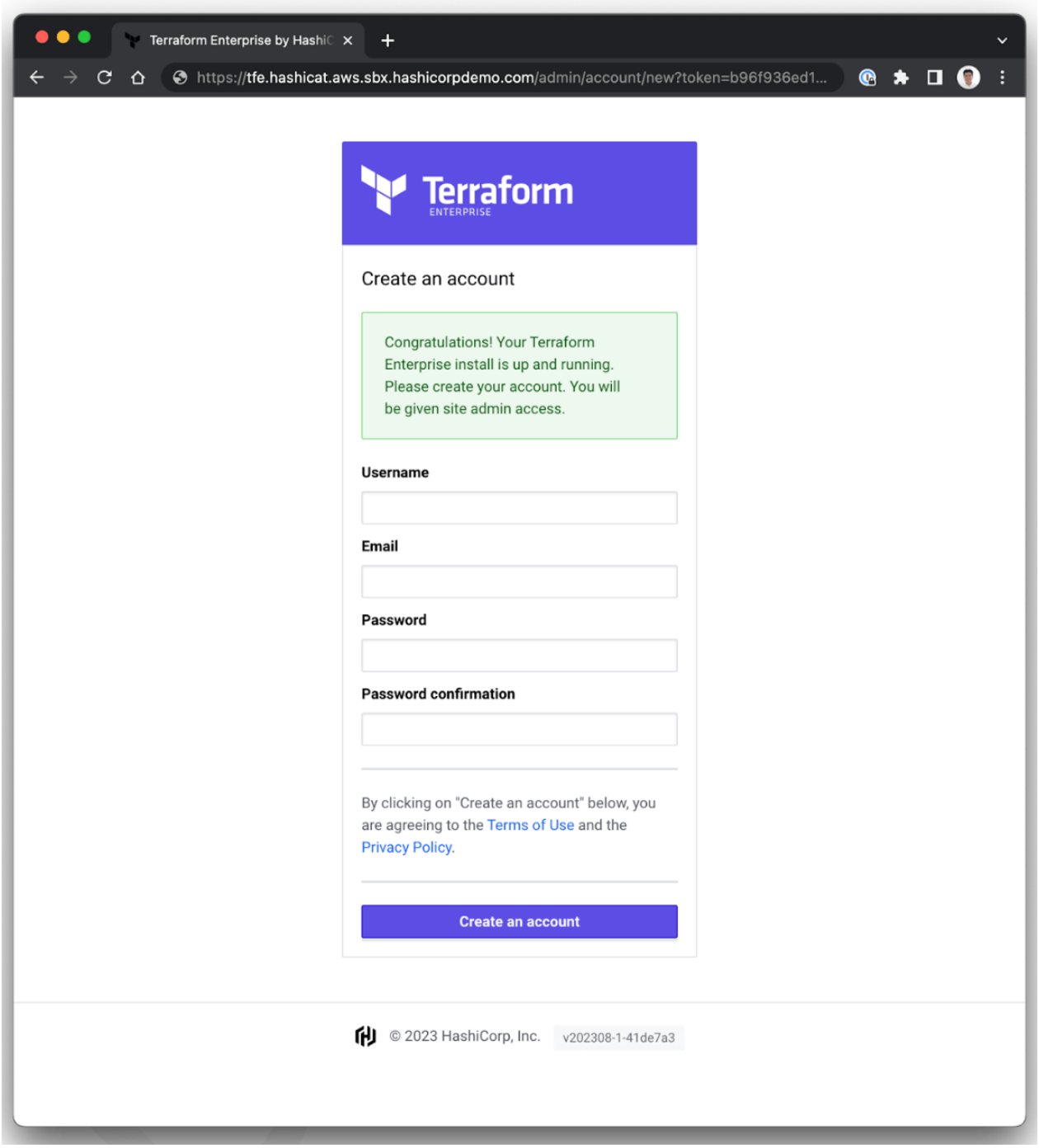


Figure 4: Login page

When the URL loads, the operator sees a form. The first user created in Terraform Enterprise becomes the site administrator. The only reason to complete this form is when the deployment team is responsible for post-installation tasks of the Terraform Enterprise deployment team.

8 Air-gapped configuration

This page assumes readers are familiar with an existing Terraform Enterprise deployment and architecture. Terraform Enterprise supports deployment in network-restricted environments, commonly referred to as *air-gapped*, where outbound Internet access is limited or prohibited. This page covers the five primary components that require internal hosting in an air-gapped environment, the design decisions you need to make before deployment begins, and the operational commitments that come with maintaining those components over time.

8.1 Network egress

Terraform Enterprise makes outbound connections to several public endpoints at startup, during runs, and when optional integrations are active. In an air-gapped environment, each endpoint must either be replaced with an internal equivalent, allowed through as a deliberate firewall exception, or disabled if the associated feature is not in use.

Destination	Purpose	Airgap treatment
`images.releases.hashicorp.com`	Application container image	Stage in internal registry; one-time egress during initial image staging
`s3-r-w.us-east-1.amazonaws.coms3-r-w.us-west-2.amazonaws.coms3-r-w.eu-central-1.amazonaws.coms3-r-w.eu-west-1.amazonaws.com`	CDN origin for container image layer traffic	Required during initial image staging. These S3 endpoints are globally routable and subject to change. Verify the current list in the network configuration reference

Destination	Purpose	Airgap treatment
`helm.releases.hashicorp.com`	Helm chart for Kubernetes deployments	Pull the chart to an internal Helm repository for fully air-gapped Kubernetes installs
`releases.hashicorp.com`	Terraform CLI and Sentinel binaries	Stage on internal HTTPS artifact server
`releases.hashicorp.com/tfc-agent`	HCP Terraform agent image and binaries	Stage in internal registry/artifact server; set `TFC_AGENT_AUTO_UPDATE=disabled` on agent hosts
`registry.terraform.io`	Terraform provider binaries and public module registry	Replace with internal network mirror or Terraform Enterprise private registry
`reporting.hashicorp.services`	Automated license entitlement reporting	Deliberate egress exception, or route through an authenticated internal proxy; see license reporting
`yy0ffni7mf-dsn.algolia.net`	Terraform Registry search index	Disable public registry search or allow as a deliberate egress exception
VCS provider endpoints	Webhook delivery and API access for VCS-driven workflows	Internal VCS: no egress. SaaS VCS (GitHub.com, GitLab.com, Azure DevOps): deliberate egress exception
SAML / OIDC identity provider	User authentication	Internal IdP: no egress. SaaS IdP (Okta, Entra ID): deliberate egress exception

Destination	Purpose	Airgap treatment
`api.pricing.us-east-1.amazonaws.com` `cloudbilling.googleapis.com` `comprices.azure.com`	Real-time pricing data for cost estimation	Disable cost estimation if egress is unavailable

For the complete list of ports and all network configuration variables, see Configure network settings in the Terraform Enterprise deployment documentation.

8.2 Five components to stage internally

Terraform Enterprise depends on five external sources that must have internal equivalents before you can operate in an air-gapped environment.

Component	Default public source	When it is fetched
Terraform Enterprise application container image	`images.releases.hashicorp.com`	At every Terraform Enterprise container startup and host spawn
Terraform CLI binaries	`releases.hashicorp.com/terraform`	Before each workspace plan or apply run
Sentinel CLI binaries	`releases.hashicorp.com/sentinel`	Before each workspace run that enforces Sentinel policies
Terraform provider binaries	`registry.terraform.io`	During `terraform init` inside each run
HCP Terraform agent container image and binaries	`releases.hashicorp.com/tfc-agent`	When provisioning or updating agent pool hosts; at agent startup if `TFC_AGENT_AUTO_UPDATE` is not disabled

Each component has a different fetch timing, which shapes your staging priorities:

- Stage the **application container image first** — Terraform Enterprise cannot start without it.
- Stage **CLI binaries before enabling workspaces** — Terraform Enterprise starts successfully, but every run fails until at least one Terraform version is reachable.
- Stage **provider binaries progressively** — you can add providers and versions after Terraform Enterprise is running and workspaces are in use.
- Stage the **agent image or binaries before creating agent pools** — *ONLY* required if you intend to horizontally scale or broach network boundaries. Runs that target an agent pool fail immediately if the agent cannot start. Set `TFC_AGENT_AUTO_UPDATE=disabled` to prevent the agent from attempting a runtime pull from the Internet.

8.3 Application container image

8.3.1 Staging the image

Pull the Terraform Enterprise image from HashiCorp's registry on an Internet-connected workstation, then push it to your internal container registry. Authentication to `images.releases.hashicorp.com` requires your Terraform Enterprise license file (`.hclic`).

```
# Authenticate to HashiCorp's registry using the license file as the password
echo "<TFE_LICENSE_CONTENTS>" | docker login \
  --username terraform \
  --password-stdin \
  images.releases.hashicorp.com

# Pull the target version
docker pull images.releases.hashicorp.com/hashicorp/terraform-enterprise:<vYYYYMM-#>

# Tag and push to your internal registry
docker tag \
  images.releases.hashicorp.com/hashicorp/terraform-enterprise:<vYYYYMM-#> \
  <internal-registry-host>/<repo>:<vYYYYMM-#>

docker push <internal-registry-host>/<repo>:<vYYYYMM-#>
```

8.3.2 Configuring TFE to use the internal registry

Docker or Podman on VM (HVD module):

```
tfe_image_repository_url      = "internal-registry.example.com"
tfe_image_name                = "example/terraform-enterprise"
tfe_image_tag                 = "v202505-1"
tfe_image_repository_username = "registry-user"
tfe_image_repository_password = "registry-password"
```

Kubernetes (Helm):

```
image:
  repository: internal-registry.example.com
  name: example/terraform-enterprise
  tag: v202505-1
```

Create the image pull secret in the TFE namespace before running the Helm install:

```
kubectl create secret docker-registry terraform-enterprise \
  --namespace <TFE_NAMESPACE> \
  --docker-server=internal-registry.example.com \
  --docker-username=registry-user \
  --docker-password=registry-password
```

8.3.3 TLS and proxy requirements

- The internal registry must serve HTTPS on port 443. Terraform Enterprise does not support HTTP-only registries.
- If the registry uses a certificate signed by an internal CA, set `TFE_TLS_CA_BUNDLE_FILE` to the path of your CA bundle on the Terraform Enterprise host.
- Add the internal registry hostname to `TFE_NO_PROXY`. Routing registry traffic through an external proxy adds unnecessary complexity and latency.
- Confirm that the container runtime daemon on each Terraform Enterprise host also trusts the internal CA — a daemon-level pull failure occurs before TFE's own configuration is evaluated.

For the full configuration reference, see Terraform Enterprise deployment configuration reference.

8.3.4 Terraform run pipeline (optional)

Terraform Enterprise ships with a version of the terraform agent image. This image is used by the terraform run pipeline to execute terraform runs in an isolated environment. If you need the

custom agent image on the core platform, you can enable it with the `tfe_run_pipeline_image` configuration, which can use the custom agent images described below.

8.4 Client binaries

Terraform Enterprise does not bundle Terraform CLI binaries inside the application container. It fetches the appropriate binary before each workspace run. In an air-gapped environment, you replace the default `releases.hashicorp.com` endpoint with an internal HTTPS server.

From an Internet-connected workstation, download the `linux_amd64` build and its SHA-256 checksum for each Terraform version your workspaces require:

```
TERRAFORM_VERSION="1.9.8"
BASE_URL="https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}"

curl -O "${BASE_URL}/terraform_${TERRAFORM_VERSION}_linux_amd64.zip"
curl -O "${BASE_URL}/terraform_${TERRAFORM_VERSION}_SHA256SUMS"

# Verify checksum before staging
grep "linux_amd64" terraform_${TERRAFORM_VERSION}_SHA256SUMS | sha256sum --check
```

Upload the `.zip` file to your internal HTTPS artifact server (Artifactory, Nexus, S3-compatible storage, or a plain nginx-served directory). A custom API is not required — a static file server is sufficient.

If your run environments use ARM hosts, also download and stage `linux_arm64` builds and register them separately.

8.4.1 Registering versions in Terraform Enterprise

Do not enable a version in Terraform Enterprise before its binary is reachable at the configured URL. Terraform Enterprise marks the version as enabled and attempts to download it on the next run that targets that version. A download failure produces a confusing run error, not a clear configuration warning.

Using the Admin UI: Navigate to **Admin Settings** **Terraform Versions**, click **Add Terraform Version**, and fill in the version string, the internal URL, and the SHA-256 checksum.

Using the Admin API:

```
curl \
  --header "Authorization: Bearer $TFE_TOKEN" \
  --header "Content-Type: application/vnd.api+json" \
  --request POST \
  --data '{
    "data": {
      "type": "terraform-versions",
      "attributes": {
        "version": "1.9.8",
        "url": "https://internal-artifacts.example.com/terraform/1.9.8/terraform_1
            .9.8_linux_amd64.zip",
        "sha": "<sha256-checksum>",
        "enabled": true,
        "deprecated": false,
        "beta": false
      }
    }
  }' \
  https://<TFE_HOSTNAME>/api/v2/admin/terraform-versions
```

Using the TFE Terraform provider for version-controlled lifecycle management:

```
resource "tfe_terraform_version" "v1_9_8" {
  version    = "1.9.8"
  url        = "https://internal-artifacts.example.com/terraform/1.9.8/terraform_1
      .9.8_linux_amd64.zip"
  sha        = "<sha256-checksum>"
  enabled    = true
  deprecated = false
  beta       = false
}
```

For the full API reference, see Admin Terraform Versions API.

8.5 Sentinel CLI binaries

Sentinel CLI binaries follow the same model as Terraform CLI binaries. Terraform Enterprise fetches them from `releases.hashicorp.com/sentinel` by default. Stage them on the same internal HTTPS server and register each version using the equivalent Admin Sentinel Versions API or the `tfe_sentinel_version` Terraform provider resource.

Stage Sentinel versions for any workspaces that enforce Sentinel policy-as-code checks.

8.6 Provider binaries

Workspace runs call `terraform init`, which downloads required providers. Two approaches replace the default `registry.terraform.io` endpoint.

8.6.1 Option 1 — Network mirror protocol

Host a static HTTPS directory that implements the Terraform network mirror protocol. This is the simpler option to stand up — a static file server is sufficient, and no GPG signing is required.

Populate the mirror from an Internet-connected workstation:

```
terraform providers mirror \
  -platform=linux_amd64 \
  /path/to/local-mirror-directory
```

This command reads your `required_providers` block and downloads correctly structured provider archives. Upload the directory to your internal HTTPS file server.

Configure workspace runs to use the mirror by setting `TF_CLI_CONFIG_FILE` to a file containing:

```
provider_installation {
  network_mirror {
    url     = "https://internal-mirror.example.com/terraform-providers/"
    include = ["registry.terraform.io/*/*"]
  }
  direct {
    exclude = ["registry.terraform.io/*/*"]
  }
}
```

Set this environment variable in workspace settings or mount the file via a custom agent worker image.

8.6.2 Option 2 — Terraform Enterprise private registry

Publish provider releases to the Terraform Enterprise built-in private registry. Workspace configurations reference the private registry address in `required_providers` blocks. This option makes provider versions visible and searchable in the Terraform Enterprise web UI, but requires GPG signing for each published release and a manual publication workflow per version.

8.6.3 Choosing between the two options

Factor	Network mirror	Private registry
Terraform Enterprise UI discoverability	No	Yes
GPG signing required	No	Yes
Server-side complexity	Low (static files)	Low (TFE built-in)
Version enforcement model	Pre-staged at mirror population time	Per-published release

For the CLI reference and configuration details, see terraform providers mirror and provider installation configuration.

8.7 Additional egress considerations

Several Terraform Enterprise features introduce egress dependencies beyond the five core components.

8.7.1 SAML and single sign-on

Terraform Enterprise's SAML 2.0 integration requires connectivity to the identity provider (IdP) metadata endpoint at login time. Internal IdPs (Active Directory Federation Services, on-premises Okta, Ping Identity) typically do not require public egress. SaaS-hosted IdPs (Okta SaaS, Microsoft Entra ID) require a deliberate egress exception for Terraform Enterprise to reach the IdP's HTTPS endpoints.

8.7.2 VCS integration

Terraform Enterprise's VCS integration requires outbound API calls to the VCS provider and the ability to receive inbound webhook events. Self-managed VCS systems (GitHub Enterprise Server, GitLab self-managed, Bitbucket Data Center) typically work within the same network segment. SaaS VCS providers (GitHub.com, GitLab.com, Azure DevOps) require an outbound egress exception.

If egress to a cloud-hosted VCS is not possible, the CLI-driven or API-driven workflow provides an alternative: a CI system handles VCS communication and pushes plans and applies to Terraform Enterprise via the API, removing the need for Terraform Enterprise to connect to the VCS directly.

8.7.3 HCP Terraform agents

`tfc-agent` pools route workspace runs to isolated execution environments. When an agent handles a run, the **agent host** — not the Terraform Enterprise container — is where `terraform init`, provider downloads, and plan or apply execution occur. Every internal resource an agent needs must be reachable from agent hosts, not necessarily from the Terraform Enterprise application host.

Staging the agent

The agent ships as both an official Docker container image and a standalone Linux binary. Stage whichever you intend to use.

Container image:

```
docker pull hashicorp/tfc-agent:latest
docker tag hashicorp/tfc-agent:latest <internal-registry-host>/hashicorp/tfc-agent:
    latest
docker push <internal-registry-host>/hashicorp/tfc-agent:latest
```

Standalone binary:

Download both `tfc-agent` and `tfc-agent-core` — both binaries must reside in the same directory on the agent host or the agent will not start.

```
TFC_AGENT_VERSION="1.18.1"
BASE_URL="https://releases.hashicorp.com/tfc-agent/${TFC_AGENT_VERSION}"

curl -O "${BASE_URL}/tfc-agent_${TFC_AGENT_VERSION}_linux_amd64.zip"
curl -O "${BASE_URL}/tfc-agent_${TFC_AGENT_VERSION}_SHA256SUMS"

grep "linux_amd64" tfc-agent_${TFC_AGENT_VERSION}_SHA256SUMS | sha256sum --check
```

Upload the extracted binaries to your internal artifact server and distribute to agent hosts.

Disable auto-update — required in air-gapped environments

By default, the agent automatically updates itself to the latest minor version by downloading from `releases.hashicorp.com`. In an air-gapped environment this **must** be explicitly disabled. If `TFC_AGENT_AUTO_UPDATE` is not set, the agent attempts to reach the Internet at startup and on a background timer, producing connection errors that can cause the agent process to exit unexpectedly.

Set `TFC_AGENT_AUTO_UPDATE=disabled` on every agent host before starting the process:

```
# Docker or Podman
docker run -d \
  -e TFC_AGENT_TOKEN=<agent-pool-token> \
  -e TFC_AGENT_NAME=<agent-name> \
  -e TFC_AGENT_AUTO_UPDATE=disabled \
  <internal-registry-host>/hashicorp/tfc-agent:latest

# Standalone binary
export TFC_AGENT_TOKEN=<agent-pool-token>
export TFC_AGENT_NAME=<agent-name>
export TFC_AGENT_AUTO_UPDATE=disabled
./tfc-agent
```

When auto-update is disabled, include agent binary or image version management in the same internal staging and promotion pipeline used for Terraform CLI binaries.

Agent host requirements

Requirement	Minimum
Architecture	x86_64 or ARM64
Free disk space	4 GB
System memory	2 GB
`git` on PATH	Version 2.3 or later
`ssh` on PATH	Required for `git` over SSH

Optional but recommended for broad `local-exec` and external data source support: `curl`, `wget`, `jq`, `unzip`, `tar`, `gzip`.

The pool architecture is determined by the first agent to register. Attempting to register an agent with a different architecture than the existing pool members produces an error. All agents in a pool must share the same architecture.

Agent egress requirements

Agents have their own egress surface that is separate from Terraform Enterprise's. In a TFE deployment the following endpoints resolve to the TFE host — not to HashiCorp's cloud — but verify with your network team before setting firewall rules.

Destination	Port	Purpose	Airgap treatment
<TFE_HOSTNAME>	tcp/443	Job polling, status updates, artifact storage, private module downloads	Internal — no public egress required
<TFE_HOSTNAME>	tcp/7146	Agent RPC (request forwarding, Hold Your Own Key)	Internal — only required if request forwarding is enabled
releases.hashicorp.com	tcp/443	Agent self-update and Terraform binary downloads	Block; set TFC_AGENT_AUTO_UPDATE=disabled; serve Terraform binaries from internal artifact server
registry.terraform.io	tcp/443	Provider and public module downloads	Internal network mirror or Terraform Enterprise private registry
Cloud provider APIs	tcp/443	Terraform plan and apply execution	Must be reachable from agent hosts for the target cloud(s)

8.7.4 Custom agent images

For workflows that require additional tooling during `local-exec` provisioner scripts or external data sources, extend the staged agent image. See the custom worker image page for more information.

For the full configuration reference, see HCP Terraform Agents and Agent requirements.

8.7.5 License validation and reporting

Terraform Enterprise validates the `.hclic` license file locally at startup — no outbound connection is required for validation. License renewal requires obtaining a new `.hclic` from HashiCorp and restarting Terraform Enterprise with the updated file or environment variable.

Automated license entitlement reporting is a separate feature that sends usage data to `reporting.hashicorp.services`. This service is active by default in FDO deployments. In an air-gapped environment, configure a firewall exception for this endpoint or route it through an internal proxy. For details, see Enable automated license reports.

8.7.6 Cost estimation

When cost estimation is enabled, Terraform Enterprise calls the following endpoints to retrieve real-time pricing data:

- `api.pricing.us-east-1.amazonaws.com` (AWS)
- `cloudbilling.googleapis.com` (GCP)
- `prices.azure.com` (Azure)

If egress to these endpoints is not available, disable cost estimation in Admin Settings to prevent run failures for workspaces that target those cloud providers.

8.7.7 Summary of egress dependencies

Component or feature	Public egress required?	Notes
TFE container image — initial pull	Yes, once	Pull from `images.releases.hashicorp.com` + S3 CDN origins before internal staging
TFE container image — ongoing	No	Served from internal registry
Terraform CLI binaries — staging	Yes, once per version	Download from `releases.hashicorp.com` before staging
Terraform CLI binaries — runtime	No	Served from internal HTTPS server
Provider binaries — staging	Yes, once per version	Download from `registry.terraform.io` or GitHub Releases
Provider binaries — runtime	No	Served from internal mirror or private registry
SAML/SSO	Depends on IdP location	Internal IdP requires no public egress; SaaS IdP does
VCS integration	Depends on VCS location	Self-managed VCS requires no public egress; SaaS VCS does
HCP Terraform agents — job polling	No	Agents call TFE API on the internal network
HCP Terraform agents — provider downloads	No	Agents reach internal mirror or private registry
License validation	No	`.hclic` validated locally at startup
Automated license reporting	Yes	`reporting.hashicorp.services`; configure egress exception or internal proxy

Component or feature	Public egress required?	Notes
Registry search (Algolia)	Yes	`yy0ffni7mf-dsn.algolia.net`; disable public registry search if egress is unavailable
Cost estimation	Yes, if enabled	Three cloud pricing APIs; disable cost estimation if egress is unavailable
Support diagnostics	No	Bundles generated locally and uploaded manually

8.8 Pre-baked VM images with Packer

For environments where repeated runtime pulls are operationally unacceptable, consider baking the TFE container image (and optionally the agent image) into a base VM image using HashiCorp Packer. A temporary build VM with controlled egress pulls the required images once, seeds a local Docker Registry v2 instance, and bakes the registry data into the VM image. Runtime hosts launch from this pre-baked image and have no pull dependency at deployment time.

The build-time provisioner pattern:

```
#!/usr/bin/env bash
# Minimum skeleton — see Packer shell provisioner docs for full context
TFE_IMAGE="images.releases.hashicorp.com/hashicorp/terraform-enterprise:${TFE_VERSION
    }"
TFE_LOCAL_TAG="localhost:5000/hashicorp/terraform-enterprise:${TFE_VERSION}"

# Start a seed registry
docker run -d --name tfe-registry-seed -p 5000:5000 \
  -v /var/lib/registry:/var/lib/registry registry:2

# Authenticate, pull, retag, push to local registry
cat /tmp/tfe.hclic | docker login --username terraform \
  --password-stdin images.releases.hashicorp.com
docker pull "${TFE_IMAGE}"
docker tag "${TFE_IMAGE}" "${TFE_LOCAL_TAG}"
docker push "${TFE_LOCAL_TAG}"

# Clean up credentials and upstream tags before image capture
docker logout images.releases.hashicorp.com
rm -f /root/.docker/config.json /tmp/tfe.hclic
docker rmi "${TFE_IMAGE}" || true
```

Key security controls for Packer builds:

- Never commit the `.hclic` license file — upload it to a temporary path during the build and delete it before image capture
- Remove Docker credential files before the image is finalized
- Remove upstream image tags so the baked image contains only local registry references

For full Packer template patterns and platform-specific notes (AWS, Azure, GCP, VMware), see the Packer documentation.

8.9 Ongoing operational commitments

Air-gapped operation creates an ongoing maintenance obligation that does not exist in Internet-connected deployments.

You must re-mirror every new version you want to use. This applies to:

- Each new Terraform Enterprise version you deploy or upgrade to.
- Each new Terraform CLI version workspaces need.
- Each new provider version workspace configurations reference.

- Each new `tfc-agent` version deployed to agent pool hosts.

Automate version lifecycle management to keep this overhead manageable:

- A CI pipeline that watches `releases.hashicorp.com` and syncs new CLI binaries to the internal artifact server.
- A downstream step that calls the Admin API to register and enable the new version.
- A mirror refresh job that runs `terraform providers mirror` for any newly required provider versions and uploads the results.
- A pipeline stage that stages new `tfc-agent` images or binaries to the internal registry/artifact server and updates agent pool hosts.

Confirm TLS is in place everywhere. The internal registry, the CLI binary server, and the provider mirror all need certificates that Terraform Enterprise and your run environments trust. Missing or misconfigured CA bundles are the most common failure cause in air-gapped deployments.

8.10 Additional considerations

Terraform enterprise enables several capabilities that you should also consider when designing your air-gapped deployment.

- **Run tasks** require access to technology partner integrations or custom integrations. See Run tasks.
- **Workspace notifications** use webhooks to notify external systems about run progress and other events. See Notifications.
- **Log forwarding** — we strongly recommend using an external log forwarding solution that aligns with your existing observability solutions. See External log forwarding.
- **Integrations** — the Terraform Enterprise ecosystem features a variety of integrations to let Terraform Enterprise connect with third-party systems and platforms. See Integrations.

9 Post-installation configuration

This page covers the first-run Terraform Enterprise setup tasks that follow deployment. HCP Terraform does not require installation; HCP Terraform readers should use the Administration and User guides for shared service configuration and consumption workflows.

9.1 Post-installation tasks

An optional step at the end of automated installation of Terraform Enterprise is the creation of the Initial Admin Creation Token (IACT), but further initialization tasks complete the setup. There are two viable options for post-provision configuration in Terraform Enterprise: API scripts or using the Terraform Provider. The best practice is to use the Terraform Enterprise provider to derive state for the configuration.

9.2 Connect to a version control system

We recommend connecting HCP Terraform/Terraform Enterprise to your VCS provider to enable workflows for managing modules in the private registry, managing Sentinel or OPA policy-sets, and connecting VCS-backed Stacks and workspaces to VCS repositories.

9.2.1 GitHub app

If using GitHub, the preferred method of VCS connection is a GitHub App, for a number of reasons. A GitHub App is not tied to a specific user, does not require a personal access token, and is safe to set up as a static VCS connection with no need to rotate tokens. A GitHub App connects Terraform to GitHub, but confers no access permissions itself. Instead, the following elements govern access to repositories.

- The GitHub organization(s) housing the GitHub App.
- Individual GitHub user permissions (to use the GitHub App, Terraform users must authorize it with their GitHub credentials).

See GitHub's documentation for more detail.

Terraform Enterprise supports a single site-wide GitHub App, shared across all Terraform Enterprise organizations. To use it, create and configure it using a site administrator account.

Choose the appropriate level of visibility for your GitHub App from the following list.

- If you only have one GitHub organization, configure your GitHub App as private.
- If you have multiple organizations under the same GitHub Enterprise, create the App under your enterprise account (at the time of writing, this feature is in public preview).

- If you need access to repositories across multiple GitHub organizations, configure your GitHub App as public but do not list it on GitHub's marketplace.

HCP Terraform has a pre-configured public GitHub app which you can install in any GitHub organization(s) as required.

9.2.2 OAuth

For providers other than GitHub, authorize VCS connections using OAuth within each Terraform Enterprise organization. Do this using a manual UI workflow using an ID and secret from the VCS, or automated using the HCP Terraform and Terraform Enterprise provider using an access token from the respective VCS account. As with all static tokens, it is good practice to rotate this periodically.

OAuth authorizes Terraform Enterprise to act as a specific user or account on the VCS. The permissions of that account govern the access Terraform has to VCS organizations and repositories. For simplicity, we recommend setting up a single OAuth connection per VCS provider in each Terraform Enterprise organization. Use a service account with access to all IaC and policy-as-code repositories, to avoid any dependency on individual employee accounts.

If you require a more granular or least-privilege permissions model, you can create multiple OAuth VCS connections, each mapping a subset of VCS repositories to a subset of Terraform projects. In this case it would be a function of a landing-zone module to create these OAuth connections and project mappings on demand whenever a new project onboards.

10 Identity integration

This page covers initial identity setup for Terraform Enterprise. Ongoing team, role, and SAML management belongs in the Administration Guide.

10.1 SSO and teams

The setup of SSO for HCP Terraform is not covered in detail in this document, as configurations can vary depending on the SAML 2.0-compliant provider used.

- You can find specific instructions for popular identity providers here.

- For applicable SAML 2.0 documentation, here.

We recommend automating the creation of teams alongside projects, Stacks, and workspaces using the Terraform Enterprise provider. After creating the initial teams required, automate the addition of users within your strategic IdP platform. By adding a team attribute name (default: MemberOf) attribute within your IdP, you can automatically assign users to the created groups in the SAML configuration. You can find further details here.

Note: For user accounts used by pipelines, use `IsServiceAccount` set to true to flag these in SAML. See `isserviceaccount`. Create teams in Terraform Enterprise with the *exact* name of the group in the IdP. Do not create users manually - Terraform Enterprise creates these automatically after it processes the SAML assertion at login.

11 Security hardening

11.1 Operating system

- Primarily, we strongly recommend using operating system configurations which are compliance to the CIS benchmark for the operating system you use.
- Limit command line access to the machines to a shortlist of well-known staff, and ensure your organization's strategic SIEM/audit log reflects all access.

11.2 Application

- Use single-sign-on (SSO) with multi-factor authentication (MFA) for all users.
- The deployment configuration sets up TCP port restrictions for ingress to and egress from the Terraform Enterprise application and related services. Do not alter these restrictions except according to advice from HashiCorp support, a HashiCorp solutions or implementation engineer, or a certified HashiCorp partner.
- Enable the `Strict-Transport-Security` response header. Terraform Enterprise allows you to restrict access to the metadata endpoint from Terraform operations, preventing Terraform workspaces from reading any data from the native AWS metadata service.
- When performing a manual installation, set `restrict_worker_metadata_access` as a Docker environment variable to prevent Terraform operations from accessing the cloud instance metadata service. For additional information, see this page.

- The automated deployment configuration used in this guide restricts the application instances from accessing the AWS metadata service. Do not re-enable this.
- At the end of a deployment, there is an option to create an initial administrator for Terraform Enterprise. We recommend not creating an account and coordinating a hand-off to the operations team.

12 High availability and multi-region

This page assumes readers are familiar with an existing Terraform Enterprise deployment and architecture.

Terraform Enterprise is a single region application, even when running the `active-active` operational mode. This is due primarily to PostgreSQL not operating Active/Active cross-region. However, scaled customers require the use of multiple public or private cloud regions to support redundancy in the event of the loss of the active region and it is solely for this purpose that we provide the recommendations below. As such, if your business is in this category, the primary recommendation is to operate two concurrent instances of Terraform Enterprise in two different cloud regions (public or private).

This page therefore provides detailed guidance regarding considerations for deploying Terraform Enterprise using two public or private cloud regions. For reference below, this means using a *primary* region as the main operating region, and a *secondary* region as a disaster recovery region for business continuity if the *primary* fails.

Although there are three operational modes, this guide focuses on the `active-active` operational mode, as it covers all necessary components for DR.

> **Note**
>
> Patroni is a template for high availability (HA) PostgreSQL deployments using Python. As the documentation suggests, Patroni is a template, and has caveats. While Patroni can run natively on top of Kubernetes, use of this pattern to support Terraform Enterprise is in beta, so is not yet considered a validated design.

12.1 Primary considerations

12.1.1 Cost versus risk

Terraform Enterprise provides mission critical capabilities to the business, so the benefit of duplicating the systems' tiers in a secondary region outweighs the cost.

Nonetheless, we recommend calculating the total cost of ownership (TCO) of two Terraform Enterprise instances (one in each of two regions) and ensure your project management staff are aware of this as part of annual project finance. For comparison, we also recommend working to calculate the cost to your business were developers unable to deploy applications. This is notably pertinent if you have committed spend under an enterprise discount program. We anticipate it being easier to balance cost versus risk with these parameters known.

When calculating TCO, include the cost of geo-redundant copies of the data layer.

12.1.2 Cloud support

We recommend checking public cloud provider documentation to ensure there is support for cross-region replication for each Terraform Enterprise component between the regions you are targeting. Cloud service provisioning for this is still developing as cloud service providers add new regions to their platforms.

12.1.3 Automation

Only use automated means to deploy all infrastructure to the primary and secondary regions used for Terraform Enterprise. This provides logical reproducibility, code versioning and speed of redeployment. We recommend using cloud automation tools to replicate the data layers of the platform to the secondary region. We refer to these below.

We strongly recommend using the HVD Modules to deploy Terraform Enterprise as this ensures you mirror the resources in each region. See below for more details specific to your chosen cloud partner. Deploy each region separately and maintain state for each separately.

We do not recommend using automation for detecting *and* failing over Terraform Enterprise from one region to the other. We do recommend use of automated health checking, and also (optionally) the actual fail over process, however, putting these two together bodes less well in situations where

transient network outages occur. In such scenarios, Terraform Enterprise may fail over unnecessarily which may have undesirable consequences, as there are sometimes delays in cross-region data layer replication.

It is normal for senior management or dedicated business continuity staff to declare an outage, so we recommend hooking into this process. In a full-region failover scenario, there are likely to be a large number of applications requiring movement and this might represent noticeable network congestion between regions. This is of particular note on premise where VMware vMotion may result in delays as massive numbers of applications migrate at the same time. We recommend liaising with staff in position to manage such circumstances to ensure you are aware of where in the priority list of applications your Terraform Enterprise instances are so you can better manage failover.

12.1.4 Testing

It is of primary importance to test your region failover capability on a regular basis using a cadence in line with business policy. We recommend at least twice annually.

Document both the region failover and failback processes step-by-step in run books. Ensure that team members who did *not* write the document use it to perform the fail over and fail back testing as this ensures that the document works and is clear. This also helps train staff.

Deploy a pair of engineering Terraform Enterprise instances, one in each of the same regions that your production instances use. Mirror your engineering instances from production in terms of resources for dev/prod parity and populate them with test data - this makes for a meaningful failover test. Document the experience in the run book.

Maintain independent instances. Each instance in each environment must have its own DNS, storage, and supporting services. Avoid configuring failover environments to point back to production DNS or services.

Perform fault injection testing by using cloud service provider features or third-party tooling to simulate availability zone or regional outages for realistic failover testing. Document the experience.

12.1.5 Configuration security

Ensure the following are *securely* available to *both* environments. These may vary depending on your configuration. Include details about the locations of these data, their storage and access in

your run book and review and maintain these at regular intervals as normal business-as-usual operations.

- Terraform Enterprise license key.
- Registry credentials for pulling container images.
- TLS keys and certificates.
- The Terraform Enterprise encryption password used to access the VAULT_TOKEN at application start-up.
- Secrets to connect to PostgreSQL, Redis, and object storage.

12.2 Component-specific guidance

The Terraform Enterprise `active-active` operational mode application architecture comprises compute, object storage, database and caching layers. We detail each of these below in a multi-region context.

> **Note**
>
> This section provides guidance on the architectural details of a multi-region Terraform Enterprise deployment. For specifics of how to fail over and fail back, refer to the Backup and Restore pages of the *Terraform: Administration and User Guides for Adoption*.

12.2.1 Compute

The Terraform Enterprise `active-active` operational mode operates a stateless compute layer irrespective of whether it is running on VMs, Nomad or Kubernetes.

- Ensure VM and container images are version-controlled and available in the failover region. We recommend using Packer as the industry standard for machine image creation.
- Do not run the Terraform Enterprise application container(s) in the secondary region while the primary region is online. Specifically, the main risk of doing this is in case of the premature promotion of the database read replica (see below) to read-write, which risks database corruption.
    - Keep the compute cluster infrastructure deployed but scaled down until failover.

    - Keep the Terraform Enterprise application containers installed on the cluster at the same version in both regions and upgrade them during the same change window. Docker- and Podman-based deployments can also have a single host ready to run Terraform Enterprise on.

- For performance reasons, co-locate your primary and secondary Terraform Enterprise compute layers in the same regions as the corresponding object store and database components respectively.

12.2.2 S3-compatible storage

AWS

- Use AWS S3 cross-region replication (CRR) on the object store; this means creating one S3 bucket in each region, configured to replicate from the *primary* to the *secondary*.
- Use live replication.
- Also use S3 CRR on buckets that store Terraform Enterprise database snapshots and on the `bootstrap` buckets that store the air-gapped installation media if applicable. Doing this locates critical data local to the ASG in the respective region.
- Use bidirectional replication to synchronize data between the two regions during failover.
- AWS refers only to S3 Same-Region Replication to confer control over data sovereignty. When using CRR, select regions in line with your regulatory requirements.
- We recommend enabling Amazon S3 server access logging in both regions as this means you can refer to the logs to understand what the last object updated in the Terraform Enterprise object store was and compare this to the outage time to better ascertain the extent of lost data during the outage. Note that the HVD Modules do not enable this. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF9E8,colframe=HexColorFBEABF,coltitle=HexColor9E4B00,title=Note"} AWS confirms that 99.99% of S3-stored objects replicate within 15 minutes and, as a result, this impacts your RPO. It also means that if you need to fail over, we strongly recommend including a task in your run book to check for missing objects.If RDS/Aurora has replicated database pointers to S3 bucket objects which are not themselves replicated at the point of failover, dangling database pointers may result. This manifests as UI artifacts such as, but not limited to, missing run or state history items. Refer to the *Terraform: Administration and User Guides for Adoption* for more information.We also recommend using AWS S3 Replication Time Control to monitor S3 CRR progress, and include maintenance tasks in your run book. Although such metrics are visible in the Metrics tab in the AWS console, use CloudWatch as it writes replication metrics to the destination region.

Note

At this time, the HVD Modules do not deploy* Bidirectional cross-region replication. Refer to this page for information on adding this. * S3 Replication Time Control. This is a separate consideration depending on the criticality of Terraform Enterprise to your business.

Tip

S3 cross-region replication (CRR) can provide replicas in multiple *secondary* regions simultaneously, so it is possible to operate greater than n-1 region redundancy if required.

Azure

- Use Azure geo-zone-redundant storage (GZRS) on Terraform Enterprise object store storage accounts. GZRS provides sixteen nines of durability, maintains the recommended zone-redundant storage (ZRS) replication on objects in the *primary* region, and replicates data to the availability zones in the *secondary* region. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF5F5,colframe=HexColorFBD4D4,coltitle=HexColorC00005,title=Warnir
  Not all Azure regions support GZRS. In fact, not all Azure regions support availability zones.For GZRS to work, and thus to deploy Terraform Enterprise in multiple Azure regions in a reliable manner, deploy instances only in paired regions *which also have availability zone support*.We recommend specifically checking data integrity if deploying onto Azure using GZRS.

- Geo-redundant data transfer happens asynchronously so there is a potential for data loss.
- When planning a Terraform Enterprise deployment across multiple regions, consider that after failover, local data replication is geo-redundant storage (GRS) not GZRS. Read this document for more information.
- Use Standard general-purpose v2 storage account types only with GRS and GZRS.
- Bear in mind Azure's region pairing model with respect to geo-redundancy of the data layer. This also impacts the database layer - see below.
- If you have data sovereignty considerations, select two paired regions which also work for your company or regulator policy.
- We recommend enabling Azure Storage Analytics logs (classic) in both regions as this means you can refer to the logs to understand what the last object updated in the Terraform Enterprise object store was and compare this to the outage time to better ascertain the extent of lost data during the outage. Note that the HVD Modules do not enable this. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF9E8,colframe=HexColorFBEABF,coltitle=HexColor9E4B00,title=Note"}
  Azure confirms that use of Azure Storage Geo Priority Replication guarantees replication to the secondary region for 99.0% of blob-stored objects within 15 minutes. This therefore impacts your RPO. It also means that if you need to fail over, we strongly recommend including a task in your run book to check for missing objects.If Azure Database for PostgreSQL has replicated database pointers to Azure blob store objects which are not replicated at the point of failover, dangling database pointers may result. This manifests as UI artifacts such as, but

not limited to, missing run or state history items. Refer to the *Terraform: Administration and User Guides for Adoption* for more information.As well as being compatible with the type of storage Terraform Enterprise writes to the object store, we also recommend using Azure Storage Geo Priority Replication to increase visibility into your Blob Geo Lag. However, we strongly urge readers to refer to the preceding link for a full explanation of what this service, and its associated SLA, offers.At this time of writing however, Azure Storage Geo Priority Replication is new in the Azure API, and the `azurerm` Terraform provider cannot deploy Azure Storage Geo Priority Replication. Refer to this page for information on adding this. :::

:::

GCP

- For cross-region object store GCS bucket replication, we recommend dual-region buckets. We do not recommend multi-region buckets unless you intend to enable more than one *secondary* region.
- For geo-redundant storage, Cloud Storage data is redundant within at least one geographic place as soon as you upload it.
- Be aware of the pairs of regions present in each dual-region bucket class. If you require cross-region deployment in Europe, due to `EUR4` being `europe-north1` and `europe-west4` and the requirement to co-locate the managed instance group (MIG) in the same location as the GCS bucket, you must deploy the Terraform Enterprise compute layer into one of these two regions as well to ensure a working instance with successful cross-region replication.
- If you have data sovereignty considerations, select two regions which also work for your company or regulator policy.
- If the region choices are not possible or you require greater than n-1 region redundancy, you need to either develop independent replication or use multi-region buckets. We recommend discussing this matter with a HashiCorp solution architect during planning.
- Use Turbo Replication for the object store buckets between multiple regions.
- We recommend enabling GCS usage logs in both regions as this means you can refer to the logs to understand what the last object updated in the Terraform Enterprise object store was and compare this to the outage time to better ascertain the extent of lost data during the outage. Note that the HVD Modules do not enable this. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF9E8,colframe=HexColorFBEABF,coltitle=HexColor9E4B00,title=Note"} Google confirms that use of Turbo Replication guarantees replication across regions for 100% of blob-stored objects within *fifteen minutes*. This therefore impacts your RPO. It also means that if you need to fail over, we strongly recommend including a task in your run book to check for missing objects. Google classes this as a premium feature, and while they offer a Standard Replication option (one hour), for your mission-critical Terraform Enterprise deployments, Turbo Replication is the recommendation. Turbo Replication logically means you lose less data in the event of a region failover which costs your business less operational loss and is less stressful for operations during a failover event.

:::

VMware

- Since this guide refers to multiple availability zones and maps these zones to separate VMware data centers, multi-region deployments confer connected data centers in different countries. The architecture choices for S3-compatible storage options for the object store of private cloud deployments of Terraform Enterprise are on the Manual Install page.
- The recommendation for VMware is to deploy identical object store provision in each region and use the strategic connections your business already operates between these regions to ensure migration of Terraform Enterprise workloads during outages works effectively.
- Most VMware customers solve replication with vSAN. The recommendation is to work with your VMware team to ensure you understand what technology is in place for replicating the underlying data, and what SLA is in place regarding how long it takes to replicate a completed write operation to the secondary.
- If using Dell ECS, refer to the Dell white paper for information about ECS high availability design. We recommend you back up your ECS rig and enable cross-DC replication, then test to ensure the data is available to the application in the secondary region.

:::

12.2.3 PostgreSQL

Warning

As PostgreSQL operates in one region at a time, a split-brain scenario on the database occurs if you promote the read replica in the secondary region to read-write while the primary read-write replica is still available. This leads to data divergence which cannot be automatically reconciled.As Terraform Enterprise relies on a PostgreSQL database, this is why it primarily has a single-region application architecture. As such, you require close care in operating the product across regions. It is also why automated failover is not recommended.Irrespective of your choice of cloud service provider, in a failover scenario, you need to be irrefutably correct that the primary database replica is offline, and stays offline, before promoting the secondary region read replica to read-write.

AWS

- It is possible to offer greater than n-1 region redundancy if required due to both RDS and Aurora being able to offer read replicas in multiple secondary regions on AWS.
- Use Aurora as the RDS DBaaS solution and enable cross-region read replicas.
- Ensure you are using a version of PostgreSQL which is both supported by Terraform Enterprise and also AWS Aurora read replication.
    - The HVD Modules for AWS use the `aws_rds_global_cluster` resource, so we recommend using versions of PostgreSQL from the preceding link which Aurora *global* database supports (>= version 15.4).
    - Aurora global databases replicate cross-region in about 1 second so there is negligible expectation of database writes to the secondary not completing and if these are to lost state file objects, we advise use of `terraform import` commands or the HCL equivalent to restore lost configuration.
- We recommend monitoring Aurora replicas for writer disconnects and also consider monitoring the write-through cache and logical slots for Aurora PostgreSQL logical replication if scaling your instance (monitor for `AuroraReplicaLag` and `AuroraGlobalDBReplicationLag` metrics).
- Ensure you can access database logs and add documentation to the effect to your run book with regards accessing replica logs in the secondary region. We recommend following the details in the official AWS guide on this.
- If your business operates in a regulated market and you require database audit logs, we recommend use of the PostgreSQL Audit extension (pgAudit) with your Aurora instance and, as such, recommend following the provisions on this page. Note that the HVD does not enable pgAudit.

Azure

- Consume the Azure document on PostgreSQL geo-replication to be aware of the architectural and operational significance.
- Use read replicas in Azure Database for PostgreSQL to replicate the database to the *secondary* region.
- Refer to this matrix as part of your calculation of which paired regions to use for your multi-region deployment.
- If using Terraform to deploy the database, set `geo_redundant_backup_enabled = true` in the `azurerm_postgresql_server` resource. The HVD Modules for Azure use this parameter.
- HashiCorp and Microsoft both recommend you monitor the replication lag between regions. This is important because Azure *automatically sets your database to read only* if storage reduces below 5% availability which adversely affects Terraform Enterprise service availability. The linked document also recommends setting storage threshold alerting. We recommend implementing all monitoring and alerting recom-

We also recommend the following:

- Embed the recommendations in HashiCorp's public document regarding PostgreSQL database failover for Terraform Enterprise into your multi-region failover run book.
- The HVD Modules for Terraform Enterprise support single-region deployments as this is the primary application architecture from the HashiCorp Product team. As such, from a database perspective, focus on ensuring you have point-in-time recovery enabled on the database, maximize retention, choose the SKU conferring fastest cross-region update times, and the enablement of read replicas in the secondary region.
- Work to restore PITR-based recovery of the database to the secondary region going back to an earlier date that latest. Doing this proves that your run book can direct recovery from both a region failover and a replicated database corruption scenario.

12.2.4 Redis

- Redis does **not** require replication between regions.
- Ensure you deploy Redis in both regions and make it ready in the failover region *before* starting Terraform Enterprise. The HVD Module ensures this if used iteratively to deploy instances into both regions.

13 Observability setup

This page covers initial observability setup for Terraform Enterprise. Recurring analysis, alerting, and response belong in the Administration Guide.

13.1 Configuring data collection

In this section we'll cover how to set the data collection for:

1. Collecting metrics and logs (including audit trail logs) on Terraform Enterprise
2. Audit trail logs on HCP Terraform
3. Collecting metrics and logs on HCP Terraform agents (applicable for HCP Terraform and Terraform Enterprise)

13.1.1 Terraform Enterprise metrics and logs

Configuring metrics collection on Terraform Enterprise

Explicitly enable Terraform Enterprise metrics collection as it is not enabled by default. Do this by setting the `TFE_METRICS_ENABLE` parameter to **`true`**. Next, configure your monitoring tool of choice to periodically query the Terraform Enterprise metrics endpoint to collect and store this information.

Configuration parameter	Description
`TFE_METRICS_HTTP_PORT`	The HTTP port which exposes metrics.
`TFE_METRICS_HTTPS_PORT`	The HTTPS port which exposes metrics.

Configuration parameter	Default Value
`TFE_METRICS_HTTP_PORT`	`9090`
`TFE_METRICS_HTTPS_PORT`	`9091`

Capture metrics in two formats using the metrics endpoint URL. The table below provides the available options as well as the URL to use. We recommend capturing metrics over an encrypted connection.

Metrics Endpoint URL	Metrics format
`https://<tfe_instance>:9091/metrics`	JSON
`https://<tfe_instance>:9091/metrics?format=prometheus`	Prometheus

Terraform Enterprise computes an aggregate metric value from a 5-second sample. Terraform Enterprise keeps this value in memory for 15 seconds before flushing it. This means that if the monitoring tool pooling frequency is greater than 15 seconds (for example every 60 seconds), you may be missing information necessary to detect short-lived issues.

If you are running multiple Terraform Enterprise instances, you must collect the metrics from each deployed Terraform Enterprise instance and aggregate the information to have a global view of the Terraform Enterprise service.

Note

If you are using the `terraform-aws-tfe` module to deploy Terraform Enterprise, note that by default, it enables metrics collection (see the `enable_metrics_collection` input variable).

Configuring log collection on Terraform Enterprise

Terraform Enterprise emits logs to standard output and standard error. We recommend collecting Terraform Enterprise logs in a central location, preferably using a specialized tool that provides searching and alerting capabilities, although we support sending logs to object storage for example.

There is a limited number of supported log destinations.

Category	Supported log destinations
AWS	AWS S3, AWS CloudWatch
Microsoft Azure	Azure Blob Storage, Azure Log Analytics
Google Cloud Platform	Google Cloud Platform Cloud Logging
Specialized SaaS	Datadog, Splunk Enterprise HTTP Event Collector (HEC)
Other	Syslog, Fluent Bit or Fluentd instance

Note

The Terraform Enterprise HVD Module does not enable log forwarding (see the `log_forwarding_enabled` input variable) by default.The module has support for:1) Forwarding logs to AWS CloudWatch (using a built-in configuration template). 2) Forwarding logs to AWS S3 (using a built-in configuration template). 3) Forwarding logs to a custom destination.

Implementing audit trail on Terraform Enterprise

Terraform Enterprise generates audit logs alongside its service logs. Use the official Terraform Enterprise logs documentation as the source of truth for supported destinations, forwarding options, event types, and the current audit log format.

When forwarding Terraform Enterprise logs to a Security Information and Event Management solution (SIEM) or another centralized logging platform, parse the documented audit fields. The current documentation identifies the audit log component as `terraform-enterprise.audit` and documents event types such as authentication success, authentication failure, and CSRF violations.

13.1.2 HCP Terraform audit trail logs

HCP Terraform features an Audit Log API endpoint that you must use to collect the audit events and store them in the appropriate system. To implement this solution, you need the following.

- A method to schedule and automate the audit events collection,
- A secure storage solution to store the audit events, and
- A data lifecycle solution to correctly dispose of the audit events once they are no longer required.

> **Warning**
>
> HCP Terraform keeps audit trail records for 14 days and then discards it. Design and configure your audit trail collection solution for HCP Terraform with this in mind.

If you use a Security Information and Event Management system (SIEM), this must be the destination for those audit events. Suppose you are not using a SIEM but instead are using a centralized log management solution (Datadog, New Relic, Elastic, and so on). In that case, you must send those audit events to your centralized log management system. If neither of these solutions is available, still collect those audit events and store them securely using an object storage solution, such as AWS S3.

13.1.3 Metrics and logs on HCP Terraform agents

Configuring HCP Terraform agent metrics collection

The HCP Terraform agent binary exposes telemetry data using the OpenTelemetry protocol. This

behavior allows the user to use a standard OpenTelemetry collector to push the metrics to a monitoring solution that supports the protocol, such as Prometheus or Datadog.

Because of that, to collect telemetry data from the agent, you need to have:

- A way to deploy and operate OpenTelemetry collector(s)
- A monitoring system that can integrate with OpenTelemetry collectors

Details about the selection of such a monitoring system or the operations of OpenTelemetry collectors are beyond the scope of this document. However, we provide some guidelines regarding integrating OpenTelemetry collectors with HCP Terraform agents.

> Tip
>
> If you are a Datadog customer, instead of deploying OpenTelemetry collectors, we recommend that you configure the Datadog agent to accept gRPC OpenTelemetry Protocol (OTLP) connections and then configure your HCP Terraform agent to use the Datadog agent as a metrics destination.

Because OpenTelemetry is a push system, you must start the collector before the HCP Terraform agent. Conversely, shut down the collector after you have stopped all HCP Terraform agents using that collector. We recommend having a one to one ratio of HCP Terraform agent instance and OpenTelemetry collector for long running HCP Terraform agents as this simplifies management.

The OpenTelemetry integration tags metrics with a number of useful fields, including the agent pool ID (`agent_pool_id`) and the agent name (`agent_name`). If you do not already have a naming convention for your HCP Terraform agents, then we recommend building one, as it helps you organize your dashboard with the metrics collected from HCP Terraform agents. You can then set the agent's name at startup time using the `TFC_AGENT_NAME` environment variable or the `-name` command line option.

Configuring HCP Terraform agent log collection

If you are using HCP Terraform agents in your Terraform enter deployment then configure log collection for agents.

14 Backup and restore initial setup

> Note
>
> This page applies to Terraform Enterprise only. HCP Terraform backup and restore operations are operated by HashiCorp.

This page covers initial preparation for backup and restore. Recurring backup execution and restoration runbooks belong in the Administration Guide.

14.1 Definitions

Business continuity (BC) is a corporate capability. This capability exists whenever organizations can continue to deliver their products and services at acceptable, predefined levels whenever disruptive incidents occur. When we refer to *environments*, these map to engineering/pre-development, development, various test (system integration testing, user acceptance testing and so on) and production. Reserve dedicated engineering instances for the platform team only, for replication, backup/restore and upgrade testing.

> Note
>
> The ISO 22301 document uses business continuity rather than disaster recovery (DR). As a result, this tutorial refers to business continuity instead of disaster recovery.

Two factors heavily determine your organization's ability to achieve BC:

1. Recovery Time Objective (RTO) is the target time set for the resumption of product, service, or activity delivery after an incident. For example, if an organization has an RTO of one hour, they aim to have their services running within one hour of the service disruption.
2. Recovery Point Objective (RPO) is the maximum tolerable period of data loss during an incident. For example, if an organization has an RPO of one hour, they can tolerate the loss of a maximum of one hour's data before the service disruption.

Based on these definitions, we recommend establishing the valid RTO/RPO for your Terraform Enterprise instance and approach BC accordingly. These factors determine your backup frequency and other considerations discussed below.

In this guide:

- Reference to a public cloud *availability zone* (AZ) is equivalent to a single VMware-based *data center*.
- Reference to a public cloud *multi-availability zone* is equivalent to a *multi-data center* **VMware** deployment.
- The main AZ is the *primary*. Any other AZs in the same region are the *secondary*. The secondary *region* is a business continuity/failover region only and is not an active secondary location. Consider all availability zones equal for the purposes of illustration.

14.2 Best practices

14.2.1 Maintain the backup and restore process

When you deploy Terraform Enterprise:

- Test the backup and restoration process and measure the recovery time to ensure it satisfies your organization's RTO/RPO.
- Document the backup and restoration process.
- Arrange for staff who did *not* write this document to run a test restore using it. This measure increases confidence in the backup and restore process.
- Use a regular schedule to test the backup and restoration process to ensure the documentation is reliable. This increases familiarity with the backup and restoration process and covers situations where staff leave.

14.2.2 Manage sensitive values

For automated deployments, you must manage several sensitive values. On Kubernetes, refer to this support page for base information. The HVD deployment modules also provide further information specific to the deployment method and target platform - see below for further details. The methods below do not back up these data. We recommend HashiCorp Vault for storage of all pipeline secrets. Do not store any of these sensitive values in version control or allow them to leak into shell histories. Do not store any of these sensitive values in version control or allow them to leak into shell histories.

We recommend HashiCorp Vault for storage of all pipeline secrets.

14.2.3 Process audit logs

Audit log processing helps you identify the root cause during a data recovery incident.

Follow the guidance on this Terraform Enterprise logs resource page to aggregate and index logs from the Terraform Enterprise node(s) using a central logging platform such as Splunk, ELK, or a cloud-native solution. Use these as a diagnostic tool in the event of outage, scanning them case insensitively for `ERROR` and `FATAL` messages as part of root cause analysis.

14.2.4 Terraform Enterprise backup API

Terraform Enterprise has a Backup API but this primarily facilitates migrations from one operational mode to another. Only use the backup API for such migrations. Use cloud-native tooling instead for day-to-day backup and recovery on public cloud, and standard approaches for on-premise deployments as detailed below.

14.3 Initial considerations

The following recommendations improve your security posture, reduce the effort required to maintain an optimal Terraform Enterprise instance, and speed up deployment time during a restoration. Apply all relevant points in the list below for your target platform (VM-based or Kubernetes-based deployments).

- Harden server images using CIS benchmarking.
- Secure Terraform Enterprise deployments on both single-tenant and shared Kubernetes clusters. If using Kubernetes clusters, lock down all APIs properly using automated configuration capabilities such as Ansible.
- If using VM-based deployments of Terraform Enterprise, use single-tenant, immutable instances using automation by repaving instances with patched images rather than patching them in place. This process requires you to maintain the setup configuration in the code used to deploy the system and is out of the scope of this document.
- Remove all unnecessary packages from the operating system.
- Ensure to store deployment configuration in a version control system and use git best practices on the repositories used to version the code.

15 Disaster recovery initial setup

> **Note**
>
> This page applies to Terraform Enterprise only. HCP Terraform disaster recovery is operated by HashiCorp.

This page captures the initial disaster-recovery considerations that should be validated before go-live. Recurring failover and failback operations belong in the Administration Guide.

15.1 Detection of need

We recommend deep observability provision around Terraform Enterprise both in terms of live metrics consumption and processing the diagnostics API. Further, we strongly recommend connecting alerts to these metrics in case `200` responses are not experienced.

These provisions afford the Terraform Enterprise platform owner the ability to keep a close eye on the health of the platform. It is always better to be able to notify your user base of an outage situation than experience them notifying you.

On receiving a system health alert, the platform team can understand the level of outage and notify senior management accordingly, but we suggest that the decision to failover requires human confirmation.

16 Next steps

Please review the Terraform Administration Guide in the HVD series.

16.1 Product support

Please ensure that all members of your Terraform Enterprise administration team do the following.

- Ensure access to a HashiCorp support portal account.
- Are setup to be an authorized technical contact for your organization in the HashiCorp support portal. Authorized technical contacts are the members of your team permitted to open support tickets on behalf of your company.

- Understand the guidance we prescribe through support for Terraform Enterprise documentation as to how to open a support ticket with HashiCorp Support, as well as generating and uploading a support bundle, when applicable. Please see the Terraform Enterprise support webpage for more details.
- Are aware of what support plan level your company has purchased to manage expectations of response times, as well as the definitions in our documentation for each selectable severity level for a ticket when opening one. Please see the Customer Success Enterprise Support web page for severity definitions.

17 Appendix

17.1 AWS IAM policy

Sample AWS IAM Policy for S3 Bucket.

```
{
  "Statement": [
    {
      "Action": [
        "s3:PutObject",
        "s3:ListBucketVersions",
        "s3:ListBucket",
        "s3:GetObject",
        "s3:GetBucketLocation",
        "s3:DeleteObject"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-logging-us-east-2/*",
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-logging-us-east-2",
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-bootstrap-us-east-2/*",
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-bootstrap-us-east-2",
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-app-us-east-2/*",
        "arn:aws:s3:::hashicat-12a345-tfe-westeros-app-us-east-2"
      ],
      "Sid": "InteractWithS3"
    },
    {
      "Action": [
        "kms:ReEncrypt*",
        "kms:GenerateRandom",
        "kms:GenerateDataKey*",
        "kms:Encrypt",
        "kms:DescribeKey",
        "kms:Decrypt"
      ],
      "Effect": "Allow",
      "Resource": "arn:aws:kms:us-east-2:000000000000:key/42845c0a-d750-4a1e-b505-26
         a76ebf0035",
      "Sid": "ManagedKmsKey"
    },
```

```
    {
      "Action": "secretsmanager:GetSecretValue",
      "Effect": "Allow",
      "Resource": [
        "arn:aws:secretsmanager:us-east-2:000000000000:secret:hashicat-12a345-tfe-
           license-l7ecol",
        "arn:aws:secretsmanager:us-east-2:000000000000:secret:hashicat-12a345-
           enc_password-test-h5Vl8l",
        "arn:aws:secretsmanager:us-east-2:000000000000:secret:hashicat-12a345-
           console_password-test-B9uhcm",
        "arn:aws:secretsmanager:us-east-2:000000000000:secret:hashicat-12a345-
           cert_pem_public-E3PNej",
        "arn:aws:secretsmanager:us-east-2:000000000000:secret:hashicat-12a345-
           cert_pem_private-h5Vl8l"
      ],
      "Sid": "RetrieveSecrets"
    },
    {
      "Action": [
        "logs:PutRetentionPolicy",
        "logs:PutLogEvents",
        "logs:DescribeLogStreams",
        "logs:DescribeLogGroups",
        "logs:CreateLogStream",
        "logs:CreateLogGroup"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:logs:us-east-2:000000000000:log-group:hashicat-12a345-tfe-log-group
           :*",
        "arn:aws:logs:us-east-2:000000000000:log-group:hashicat-12a345-tfe-log-group"
      ],
      "Sid": "WriteToCloudWatchLogs"
    },
    {
      "Action": [
        "ec2:DescribeVolumes",
        "ec2:DescribeTags",
        "cloudwatch:PutMetricData"
      ],
      "Effect": "Allow",
      "Resource": "*"
    },
```

```
    {
      "Action": "autoscaling:CompleteLifecycleAction",
      "Condition": {
        "StringEquals": {
          "autoscaling:ResourceTag/asg-hook": "hashicat-12a345-us-east-2-tfe-asg-hook
              "
        }
      },
      "Effect": "Allow",
      "Resource": "*",
      "Sid": "ASGHook"
    }
  ],
  "Version": "2012-10-17"
}
```

18 Changelog

18.1 2026-08

- Restructured legacy Terraform HVD content into the HVD 2.0 Terraform Installation Guide as part of the Installation / Administration / User guide migration.

18.2 2026-01

- Fixed heading formatting in tabbed content for PDF generation
- Update meta descriptions
- Updated credits and contributors

18.3 2025-12

- Fixed typos and documentation errors
- Update title casing for disaster recovery document
- Add terraform enterprise failover recommendations
- Fix incorrect verbiage re ARM support for Terraform Enterprise

18.4 2025-11

- Revise PostgreSQL and Redis descriptions in architecture guide
- Fixed typos and documentation errors

18.5 2025-06

- Use consistent guide names and descriptions

18.6 2025-05

- Removed document structure section

18.7 2025-04

- Patch Redis Sentinel examples
- Update security hardening guidance
- Update private cloud installation guidance

18.8 2025-03

- Simplify OS recommendation

18.9 2025-02

- Fixed typos and documentation errors

18.10 2025-01

- Update installation documentation
- Fix machine types for VM+container runtime based on 2048 MiB worker

18.11 2024-04

- Updated Terraform Cloud to HCP Terraform

18.12 2023-10

- GA Release

19 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Buxton
- Adam Cavaliere
- Adeel Ahmad
- Alex Basista
- Amy Brown
- Anna Chen
- Brandon Ferguson
- Chris Steinmeyer
- Chris Stella
- CJ Obermaier
- Emmanuel Rousselle
- Fraser Pollock
- Jenna Degaust
- Judith Malnick
- Kalen Arndt
- Kranthi Katepalli
- Kyle Seneker
- Mark Lewis
- Michelle Roberts

- Prakash Manglanathan
- Purna Mehta
- Randy Keener
- Richard Russell
- Salil Subbakrishna
- Sean Doyle
- Sean Walker
- Simon Lynch
- Zeeshan Ratani